UBND TP. HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC SÀI GÒN

KHOA TOÁN - ỨNG DỤNG

BÀI BÁO CÁO

TÍNH TOÁN THÔNG MINH

<u>Đề tài</u>: Xây dựng thuật toán Ant Colony Optimization cho bài toán tìm đường đi ngắn nhất trên đồ thị

Giảng viên hướng dẫn: TS. Huỳnh Minh Trí

Nhóm thực hiện đề tài: 09

Họ và tên	MSSV
Phạm Hoàng Tiến	3123580051
Thạch Ngọc Thảo	3123580046
Nguyễn Thái Tú	3123580058
Biện Hữu Khang	3123580017

Thành Phố Hồ Chí Minh, tháng 4 năm 2026

NHẬN XÉT VÀ CHẤM ĐIỂM CỦA GIẢNG VIÊN

Họ và tên giảng viên:

Tên đề tài:

Nội dung nhận xét:

.....

.....

.....

.....

.....

.....

.....

.....

.....

Điểm:

Bằng số :.....

Bằng chữ:.....

GIẢNG VIÊN CHẤM

(Ký, ghi rõ họ tên)

BẢNG PHÂN CÔNG NHIỆM VỤ

STT	Họ và Tên	Tỉ lệ tham gia (%)
1	Phạm Hoàng Tiến	25%
2	Thạch Ngọc Thảo	25%
3	Nguyễn Thái Tú	25%
4	Biện Hữu Khang	25%

LỜI CẢM TẠ

Trước hết, nhóm chúng em xin bày tỏ lòng biết ơn sâu sắc đến thầy Huỳnh Minh Trí – giảng viên phụ trách môn Tính toán thông minh, đã tận tình giảng dạy, hướng dẫn và truyền đạt cho chúng em những kiến thức nền tảng cũng như chuyên sâu trong suốt quá trình học tập.

Trong quá trình học, thầy không chỉ giảng dạy một cách tận tình, chi tiết và dễ hiểu mà còn luôn tạo điều kiện để sinh viên tiếp cận vấn đề theo hướng tư duy logic, khoa học và thực tiễn. Những bài giảng của thầy đã giúp chúng em có cái nhìn rõ ràng hơn về lĩnh vực tính toán thông minh, đặc biệt là các phương pháp và thuật toán tối ưu hiện đại. Đây cũng chính là nền tảng quan trọng để nhóm chúng em thực hiện đề tài "Xây dựng thuật toán Ant Colony Optimization cho bài toán tìm đường đi ngắn nhất trên đồ thị".

Mặc dù nhóm đã cố gắng tìm hiểu, nghiên cứu và hoàn thiện bài báo cáo một cách tốt nhất, tuy nhiên do kiến thức và kinh nghiệm thực tế còn hạn chế nên bài làm khó tránh khỏi những thiếu sót nhất định. Nhóm chúng em rất mong nhận được sự góp ý, nhận xét và hướng dẫn thêm từ thầy để bài báo cáo được hoàn thiện hơn.

Cuối cùng, nhóm chúng em xin kính chúc thầy Huỳnh Minh Trí cùng quý thầy cô thật nhiều sức khỏe, hạnh phúc và thành công trong công tác giảng dạy.

MỤC LỤC

LỜI CẢM TẠ **1**

DANH SÁCH CÁC HÌNH **5**

DANH SÁCH CÁC BẢNG **5**

DANH SÁCH TỪ VIẾT TẮT **5**

MỞ ĐẦU **6**

1. Lý do chọn đề tài 6

2. Mục tiêu của đề tài 6

3. Phạm vi thực hiện 7

4. Phương pháp thực hiện 7

5. Cấu trúc báo cáo 7

CHƯƠNG 1: CƠ SỞ LÝ THUYẾT **9**

1.1. Tổng quan về tính toán thông minh 9

1.2. Bài toán tìm đường đi ngắn nhất trên đồ thị 9

1.2.1. Khái niệm đồ thị 10

1.2.2. Đồ thị có trọng số 10

1.2.3. Đường đi và đường đi ngắn nhất 10

1.3. Giới thiệu thuật toán Ant Colony Optimization 11

1.3.1. Ý tưởng đàn kiến trong tự nhiên 11

1.3.2. Khái niệm pheromone 12

1.3.3. Cơ chế chọn đường đi 12

1.3.4. Cơ chế cập nhật pheromone 12

1.4. Khả năng áp dụng ACO cho bài toán đường đi ngắn nhất 13

1.5. Kết luận chương 1 14

CHƯƠNG 2: XÂY DỰNG THUẬT TOÁN **15**

2.1 Phát biểu bài toán 15

2.2. Mô hình bài toán trên đồ thị có trọng số 16

2.3. Lựa chọn mô hình ACO cho đề tài 17

2.4. Các đại lượng và tham số của thuật toán 17
2.4.1. Ma trận pheromone 17
2.4.2. Thông tin heuristic 18
2.4.3. Tập đỉnh có thể lựa chọn 18
2.4.4. Các tham điều khiển 19
2.5. Xây dựng lời giải bằng hành vi của từng cá thể kiến 19
2.6. Đánh giá chất lượng lời giải 20
2.7. Cơ chế cập nhật pheromone 21
2.7.1. Bay hơi pheromone 21
2.7.2. Bổ sung pheromone 21
2.7.3. Ý nghĩa của cơ chế cập nhật 22
2.8. Quy trình thực hiện của thuật toán 22
2.9. Phân tích vai trò của các tham số 24
2.9.1. Ảnh hưởng của α 24
2.9.2. Ảnh hưởng của β 24
2.9.3. Ảnh hưởng của ρ 24
2.9.4. Ảnh hưởng của số lượng kiến và số vòng lặp 25
2.9.5. Liên hệ với phần thực nghiệm 25
2.10. Phân tích độ phức tạp và đặc điểm của thuật toán 25
2.11. Kết luận chương 2 26
CHƯƠNG 3: CÀI ĐẶT CHƯƠNG TRÌNH VÀ THỰC NGHIỆM 27
3.1. Môi trường cài đặt 27
3.2. Công cụ và thư viện sử dụng 28
3.3. Cấu trúc chương trình 29
3.4. Cài đặt các chức năng chính 31
3.4.1. Nhập dữ liệu đồ thị 32
3.4.2. Khởi tạo pheromone 33
3.4.3. Mô phỏng quá trình di chuyển của kiến 34

3.4.4. Cập nhật pheromone 35
3.4.5. Tìm đường đi tốt nhất 36
3.5. Dữ liệu thử nghiệm 37
3.6. Kết quả chạy chương trình 38
3.7. Nhận xét kết quả 41
3.8. Kết luận chương 3 42
CHƯƠNG 4: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 44
4.1. Kết quả đạt được 44
4.2. Hạn chế của chương trình 45
4.3. Hướng phát triển 45
TÀI LIỆU THAM KHẢO 47
PHỤ LỤC 48

DANH SÁCH CÁC HÌNH

HÌNH **Trang**

Hình 3.1. Cấu trúc tổng quát của chương trình mô phỏng thuật toán ACO.....**30**

Hình 3.2 Sơ đồ các chức năng chính của chương trình.....**32**

Hình 3.3 Đồ thị được xây dựng từ dữ liệu đầu vào.....**38**

Hình 3.4. Kết quả đường đi tốt nhất do thuật toán ACO tìm được **39**

Hình 3.5. Biểu đồ hội tụ của thuật toán ACO **39**

Hình 3.6. Biểu đồ so sánh kết quả giữa ACO và Dijkstra.....**40**

Hình 3.7. Minh họa đường đi ngắn nhất tìm được trên đồ thị **41**

DANH SÁCH CÁC BẢNG

BẢNG **Trang**

Bảng 3.1. Mô tả ngắn các thành phần chính của chương trình **30**

Bảng 3.2. Các tham số khởi tạo của thuật toán ACO.....**34**

Bảng 3.3. Ma trận trọng số của đồ thị dùng trong thực nghiệm.....**37**

Bảng 3.4. So sánh kết quả giữa ACO và Dijkstra **40**

DANH SÁCH TỪ VIẾT TẮT

Từ viết tắt	**Nghĩa tiếng Anh**	**Nghĩa tiếng Việt**
ACO	Ant Colony Optimization	Tối ưu hóa đàn kiến
AS	Ant System	Hệ kiến
AI	Artificial Intelligence	Trí tuệ nhân tạo
TSP	Traveling Salesman Problem	Bài toán người du lịch

MỞ ĐẦU

1. Lý do chọn đề tài

Trong thực tiễn, bài toán tìm đường đi ngắn nhất trên đồ thị là một trong những bài toán quan trọng và có tính ứng dụng cao, xuất hiện trong nhiều lĩnh vực như giao thông, mạng máy tính, logistics, định tuyến dữ liệu và tối ưu hóa vận tải. Việc xác định được tuyến đường ngắn nhất không chỉ giúp tiết kiệm thời gian di chuyển mà còn góp phần giảm chi phí và nâng cao hiệu quả hoạt động của hệ thống. Chính vì vậy, bài toán này từ lâu đã thu hút sự quan tâm của nhiều nhà nghiên cứu trong lĩnh vực khoa học máy tính và tối ưu hóa.

Bên cạnh các thuật toán kinh điển như Dijkstra hay Bellman-Ford, các phương pháp tối ưu hóa mô phỏng tự nhiên ngày càng cho thấy nhiều ưu điểm trong việc giải quyết các bài toán phức tạp. Một trong những phương pháp nổi bật là thuật toán Ant Colony Optimization, được xây dựng dựa trên hành vi tìm kiếm thức ăn và định hướng đường đi của đàn kiến trong tự nhiên. Với khả năng mô phỏng quá trình tìm kiếm dựa trên xác suất và cơ chế cập nhật pheromone, ACO là một hướng tiếp cận thú vị, hiệu quả và có tính ứng dụng cao trong các bài toán tối ưu tổ hợp.

Xuất phát từ những lý do trên, nhóm chúng em lựa chọn đề tài "Xây dựng thuật toán Ant Colony Optimization cho bài toán tìm đường đi ngắn nhất trên đồ thị" nhằm tìm hiểu sâu hơn về nguyên lý hoạt động của thuật toán, đồng thời vận dụng kiến thức đã học để giải quyết một bài toán có ý nghĩa thực tiễn.

2. Mục tiêu của đề tài

Đề tài được thực hiện với mục tiêu trước hết là tìm hiểu cơ sở lý thuyết liên quan đến bài toán tìm đường đi ngắn nhất và thuật toán Ant Colony Optimization. Trên cơ sở đó, nhóm tiến hành phân tích cơ chế hoạt động của thuật toán ACO, đặc biệt là cách thức lựa chọn đường đi dựa trên pheromone và heuristic, cũng như quy trình cập nhật pheromone sau mỗi vòng lặp.

Từ phần nghiên cứu lý thuyết, nhóm xây dựng mô hình thuật toán ACO để áp dụng vào bài toán tìm đường đi ngắn nhất trên đồ thị có trọng số. Đồng thời, nhóm tiến hành cài đặt chương trình bằng Python nhằm mô phỏng quá trình hoạt động của thuật toán và kiểm chứng kết quả trên dữ liệu thử nghiệm. Thông qua quá trình thực hiện, đề tài còn hướng tới việc đánh giá hiệu quả của phương pháp, chỉ ra những ưu điểm cũng như những hạn chế còn tồn tại của thuật toán trong thực tế.

3. Phạm vi thực hiện

Đề tài tập trung nghiên cứu và triển khai thuật toán Ant Colony Optimization cho bài toán tìm đường đi ngắn nhất trên đồ thị có trọng số. Trong phạm vi thực hiện, đồ thị được biểu diễn dưới dạng phù hợp để thuận tiện cho việc xử lý và mô phỏng, đồng thời các cạnh của đồ thị mang trọng số dương. Thuật toán được áp dụng để tìm một đường đi tối ưu từ đỉnh nguồn đến đỉnh đích trong phạm vi đồ thị đầu vào đã cho.

Do giới hạn về thời gian và kiến thức thực hiện, đề tài chủ yếu dừng ở mức mô phỏng thuật toán trên các đồ thị có kích thước nhỏ đến trung bình nhằm minh họa nguyên lý hoạt động của ACO. Đề tài chưa đi sâu vào các hướng mở rộng như tối ưu đa mục tiêu, xử lý đồ thị quy mô lớn, tích hợp với hệ thống thực tế hoặc triển khai song song để tăng tốc độ xử lý.

4. Phương pháp thực hiện

Để thực hiện đề tài, nhóm đã kết hợp nhiều phương pháp khác nhau. Trước hết, nhóm tiến hành nghiên cứu tài liệu để tìm hiểu các kiến thức nền tảng về lý thuyết đồ thị, các thuật toán tìm đường đi ngắn nhất và nguyên lý hoạt động của Ant Colony Optimization. Tiếp theo, nhóm phân tích sâu các thành phần cốt lõi của thuật toán như xác suất lựa chọn đỉnh tiếp theo, vai trò của pheromone, cơ chế bay hơi và quy tắc cập nhật pheromone sau mỗi vòng lặp.

Sau khi nắm vững cơ sở lý thuyết, nhóm xây dựng mô hình thuật toán phù hợp với bài toán đặt ra và tiến hành cài đặt chương trình bằng ngôn ngữ Python. Trong quá trình cài đặt, nhóm sử dụng các thư viện hỗ trợ để biểu diễn đồ thị, mô phỏng sự di chuyển của kiến và theo dõi kết quả tìm kiếm. Cuối cùng, nhóm tiến hành chạy thử trên các bộ dữ liệu khác nhau, so sánh kết quả và đánh giá mức độ hiệu quả của thuật toán, từ đó rút ra nhận xét cho phần thực nghiệm và kết luận.

5. Cấu trúc báo cáo

Báo cáo được xây dựng gồm các phần chính nhằm trình bày đầy đủ quá trình nghiên cứu và thực hiện đề tài. Phần mở đầu giới thiệu lý do chọn đề tài, mục tiêu, phạm vi thực hiện, phương pháp nghiên cứu và cấu trúc tổng thể của báo cáo.

Chương 1 trình bày cơ sở lý thuyết liên quan đến tính toán thông minh, bài toán tìm đường đi ngắn nhất trên đồ thị và thuật toán Ant Colony Optimization.

Chương 2 tập trung xây dựng mô hình thuật toán, trình bày các tham số, công thức xác suất lựa chọn đường đi, cơ chế cập nhật pheromone và các bước thực hiện thuật toán.

Chương 3 giới thiệu quá trình cài đặt chương trình, môi trường thực hiện, công cụ sử dụng, dữ liệu thử nghiệm và kết quả thu được. Cuối cùng, phần kết luận và hướng phát triển tổng hợp những kết quả đã đạt được, nêu ra những hạn chế còn tồn tại và đề xuất hướng nghiên cứu tiếp theo cho đề tài.

CHƯƠNG 1: CƠ SỞ LÝ THUYẾT

1.1. Tổng quan về tính toán thông minh

Trong những năm gần đây, tính toán thông minh đã trở thành một lĩnh vực quan trọng trong khoa học máy tính, đặc biệt trong bối cảnh các bài toán ngày càng phức tạp và khó giải bằng các phương pháp truyền thống. Khác với các thuật toán mang tính xác định rõ ràng, tính toán thông minh tập trung vào việc mô phỏng các quá trình trong tự nhiên hoặc hành vi thông minh của con người nhằm tìm ra lời giải gần tối ưu trong thời gian hợp lý.

Một số hướng tiếp cận phổ biến trong lĩnh vực này có thể kể đến như mạng nơ-ron nhân tạo, thuật toán di truyền, bầy đàn (swarm intelligence) hay logic mờ. Điểm chung của các phương pháp này là khả năng thích nghi, học hỏi và xử lý tốt trong môi trường có nhiều biến động hoặc không chắc chắn.

Trong đó, các thuật toán thuộc nhóm swarm intelligence – lấy cảm hứng từ hành vi tập thể của các loài sinh vật như đàn kiến, đàn ong – đang được ứng dụng rộng rãi. Thuật toán Ant Colony Optimization (ACO) là một ví dụ tiêu biểu, mô phỏng cách đàn kiến tìm đường đi ngắn nhất từ tổ đến nguồn thức ăn. Nhờ cơ chế trao đổi thông tin gián tiếp thông qua pheromone, các cá thể đơn giản có thể cùng nhau tạo ra một hành vi thông minh ở cấp độ tập thể.

Chính những đặc điểm này khiến tính toán thông minh trở thành công cụ hữu ích trong việc giải quyết các bài toán tối ưu, trong đó có bài toán tìm đường đi ngắn nhất trên đồ thị.

1.2. Bài toán tìm đường đi ngắn nhất trên đồ thị

Bài toán tìm đường đi ngắn nhất là một trong những bài toán cơ bản và có ý nghĩa thực tiễn cao trong lý thuyết đồ thị. Mục tiêu của bài toán là xác định con đường có tổng chi phí nhỏ nhất giữa hai đỉnh trong một đồ thị. Chi phí ở đây có thể được hiểu theo nhiều cách khác nhau như khoảng cách, thời gian, chi phí vận chuyển hoặc bất kỳ đại lượng nào cần được tối ưu.

Bài toán này xuất hiện trong rất nhiều ứng dụng thực tế. Chẳng hạn, trong hệ thống giao thông, việc tìm tuyến đường ngắn nhất giúp giảm thời gian di chuyển; trong mạng máy tính, nó giúp tối ưu việc truyền dữ liệu; còn trong logistics, nó hỗ trợ xây dựng lộ trình vận chuyển hiệu quả.

Tùy theo đặc điểm của đồ thị và yêu cầu cụ thể, bài toán có thể có nhiều biến thể khác nhau như tìm đường đi ngắn nhất từ một đỉnh đến tất cả các đỉnh còn lại, hay tìm

đường đi ngắn nhất giữa mọi cặp đỉnh. Trong phạm vi đề tài này, nhóm tập trung vào việc tìm đường đi ngắn nhất giữa một đỉnh nguồn và một đỉnh đích xác định.

1.2.1. Khái niệm đồ thị

Đồ thị là một cấu trúc toán học được sử dụng để mô tả mối quan hệ giữa các đối tượng. Một đồ thị bao gồm hai thành phần chính là tập các đỉnh (vertices) và tập các cạnh (edges) nối giữa các đỉnh đó.

Mỗi đỉnh trong đồ thị biểu diễn một đối tượng hoặc một trạng thái, trong khi mỗi cạnh thể hiện mối liên kết hoặc sự tương tác giữa hai đỉnh. Tùy theo cách biểu diễn, đồ thị có thể là vô hướng hoặc có hướng. Trong đồ thị vô hướng, các cạnh không có chiều, nghĩa là mối quan hệ giữa hai đỉnh là hai chiều. Ngược lại, trong đồ thị có hướng, mỗi cạnh có một hướng xác định từ đỉnh này đến đỉnh khác.

Đồ thị là công cụ rất linh hoạt và có thể được sử dụng để mô hình hóa nhiều hệ thống trong thực tế, từ mạng xã hội, hệ thống giao thông cho đến mạng lưới máy tính.

1.2.2. Đồ thị có trọng số

Trong nhiều trường hợp, mối quan hệ giữa các đỉnh không chỉ đơn giản là có hay không, mà còn đi kèm với một giá trị cụ thể. Khi đó, ta sử dụng đồ thị có trọng số để biểu diễn. Mỗi cạnh trong đồ thị sẽ được gán một giá trị số, gọi là trọng số, thể hiện chi phí hoặc độ "nặng" của việc di chuyển qua cạnh đó.

Ví dụ, trong bài toán giao thông, trọng số có thể là khoảng cách giữa hai địa điểm; trong mạng máy tính, nó có thể là độ trễ hoặc băng thông; còn trong logistics, đó có thể là chi phí vận chuyển. Việc gán trọng số giúp bài toán trở nên thực tế hơn, đồng thời cũng làm tăng độ phức tạp trong quá trình tìm kiếm lời giải.

Trong đề tài này, đồ thị được sử dụng là đồ thị có trọng số dương, nghĩa là tất cả các giá trị trọng số đều lớn hơn 0. Điều này phù hợp với nhiều bài toán thực tế và cũng giúp đảm bảo tính ổn định cho quá trình tính toán.

1.2.3. Đường đi và đường đi ngắn nhất

Đường đi trong đồ thị là một chuỗi các đỉnh liên tiếp, trong đó mỗi cặp đỉnh liền kề được nối với nhau bởi một cạnh. Độ dài của một đường đi được xác định bằng tổng trọng số của tất cả các cạnh nằm trên đường đi đó.

Trong số tất cả các đường đi có thể tồn tại giữa hai đỉnh, đường đi ngắn nhất là đường có tổng trọng số nhỏ nhất. Đây chính là lời giải mà bài toán tìm đường đi ngắn nhất hướng tới.

Tuy nhiên, trong thực tế, việc tìm đường đi ngắn nhất không phải lúc nào cũng đơn giản, đặc biệt khi đồ thị có kích thước lớn hoặc có nhiều ràng buộc phức tạp. Chính vì vậy, ngoài các thuật toán truyền thống, các phương pháp tối ưu hóa như Ant Colony Optimization được nghiên cứu và áp dụng nhằm tìm ra lời giải hiệu quả hơn trong những trường hợp khó.

1.3. Giới thiệu thuật toán Ant Colony Optimization

Ant Colony Optimization (ACO) là một thuật toán tối ưu hóa thuộc nhóm trí tuệ bầy đàn (swarm intelligence), được đề xuất dựa trên việc mô phỏng hành vi tìm kiếm thức ăn của đàn kiến trong tự nhiên. Điểm đặc biệt của thuật toán này nằm ở chỗ, mỗi cá thể (con kiến) tuy rất đơn giản và không có khả năng "nhìn toàn cục", nhưng khi hoạt động cùng nhau lại có thể tạo ra một cơ chế tìm kiếm lời giải hiệu quả cho các bài toán tối ưu.

ACO đặc biệt phù hợp với các bài toán tổ hợp như bài toán người du lịch (TSP), bài toán định tuyến và bài toán tìm đường đi ngắn nhất. Thay vì duyệt toàn bộ không gian nghiệm như các phương pháp truyền thống, ACO sử dụng cơ chế xác suất kết hợp với việc tích lũy kinh nghiệm (thông qua pheromone) để dần dần hội tụ về lời giải tốt.

Một trong những ưu điểm nổi bật của ACO là khả năng thích nghi và tìm kiếm trong không gian lớn mà không cần thông tin đầy đủ ngay từ đầu. Tuy nhiên, để hiểu rõ cách thuật toán hoạt động, cần bắt đầu từ chính hành vi của đàn kiến trong tự nhiên.

1.3.1. Ý tưởng đàn kiến trong tự nhiên

Trong tự nhiên, khi tìm kiếm thức ăn, kiến thường di chuyển một cách gần như ngẫu nhiên. Tuy nhiên, trong quá trình di chuyển, mỗi con kiến sẽ để lại một lượng chất hóa học gọi là pheromone trên đường đi của mình. Các con kiến khác khi đi qua sẽ có xu hướng lựa chọn những con đường có nồng độ pheromone cao hơn.

Ban đầu, khi chưa có thông tin, các con kiến sẽ đi theo nhiều hướng khác nhau. Một số con sẽ tình cờ tìm được đường ngắn hơn để đến nguồn thức ăn và quay trở về tổ nhanh hơn. Do đó, lượng pheromone trên những con đường ngắn sẽ được tích lũy nhanh hơn so với các con đường dài. Theo thời gian, ngày càng nhiều kiến bị "thu hút" bởi những con đường có nhiều pheromone, và cuối cùng, cả đàn sẽ hội tụ về con đường ngắn nhất.

Điều đáng chú ý là không có con kiến nào "biết" đâu là đường ngắn nhất ngay từ đầu. Chính sự tương tác gián tiếp thông qua môi trường (pheromone) đã giúp cả hệ thống tìm ra lời giải tối ưu. Đây cũng chính là ý tưởng cốt lõi được áp dụng vào thuật toán ACO.

1.3.2. Khái niệm pheromone

Trong thuật toán ACO, pheromone được mô hình hóa như một giá trị số gắn với mỗi cạnh của đồ thị. Giá trị này thể hiện mức độ "hấp dẫn" của cạnh đó đối với các con kiến khi lựa chọn đường đi.

Ban đầu, lượng pheromone trên các cạnh thường được khởi tạo bằng nhau, thể hiện việc chưa có thông tin ưu tiên. Trong quá trình thuật toán chạy, các con kiến sẽ di chuyển qua các cạnh và cập nhật lại lượng pheromone dựa trên chất lượng của đường đi mà chúng tìm được.

Một điểm quan trọng là pheromone không chỉ tăng lên mà còn bị giảm dần theo thời gian thông qua cơ chế bay hơi. Điều này giúp tránh việc thuật toán bị "kẹt" ở một lời giải cục bộ, đồng thời duy trì sự cân bằng giữa việc khai thác (exploration) và tận dụng (exploitation) các đường đi tốt đã biết.

Nhờ cơ chế này, pheromone đóng vai trò như một dạng "bộ nhớ tập thể", giúp các thế hệ kiến sau có thể học hỏi từ kinh nghiệm của những thế hệ trước.

1.3.3. Cơ chế chọn đường đi

Khi một con kiến đang đứng tại một đỉnh trong đồ thị, nó cần quyết định sẽ đi đến đỉnh nào tiếp theo. Quyết định này không hoàn toàn ngẫu nhiên mà dựa trên một xác suất được tính toán từ hai yếu tố chính: lượng pheromone trên cạnh và thông tin heuristic (thường là nghịch đảo của khoảng cách).

Cụ thể, những cạnh có lượng pheromone cao sẽ có xác suất được chọn lớn hơn, vì chúng được xem là những con đường "tốt" dựa trên kinh nghiệm trước đó. Đồng thời, các cạnh có khoảng cách ngắn (tức heuristic cao) cũng được ưu tiên hơn.

Sự kết hợp giữa hai yếu tố này giúp thuật toán vừa tận dụng được thông tin đã có, vừa không bỏ qua khả năng khám phá các đường đi mới. Trong giai đoạn đầu, khi pheromone chưa có sự khác biệt rõ ràng, quá trình chọn đường mang tính ngẫu nhiên nhiều hơn. Tuy nhiên, khi thuật toán tiến triển, các đường đi tốt dần được củng cố, và xác suất lựa chọn sẽ nghiêng về những đường đi tối ưu.

1.3.4. Cơ chế cập nhật pheromone

Sau khi tất cả các con kiến hoàn thành hành trình của mình trong một vòng lặp, thuật toán sẽ tiến hành cập nhật lại lượng pheromone trên các cạnh. Quá trình này gồm hai bước chính: bay hơi pheromone và tăng cường pheromone.

Trước hết, một phần pheromone trên tất cả các cạnh sẽ bị giảm đi theo một hệ số nhất định. Quá trình bay hơi này giúp loại bỏ dần ảnh hưởng của các đường đi kém hiệu quả, tránh việc thuật toán bị phụ thuộc quá nhiều vào các lựa chọn trước đó.

Sau đó, các con kiến sẽ "đóng góp" thêm pheromone lên những cạnh mà chúng đã đi qua. Đặc biệt, những đường đi có chất lượng tốt hơn (tức là tổng độ dài nhỏ hơn) sẽ được tăng thêm nhiều pheromone hơn. Điều này giúp các đường đi tốt ngày càng trở nên hấp dẫn hơn trong các vòng lặp tiếp theo.

Nhờ cơ chế cập nhật này, thuật toán dần dần hội tụ về những đường đi tối ưu hoặc gần tối ưu. Sự cân bằng giữa bay hơi và tăng cường pheromone đóng vai trò rất quan trọng trong việc đảm bảo hiệu quả và độ ổn định của thuật toán.

1.4. Khả năng áp dụng ACO cho bài toán đường đi ngắn nhất

Thuật toán Ant Colony Optimization tỏ ra đặc biệt phù hợp với bài toán tìm đường đi ngắn nhất trên đồ thị, bởi bản chất của nó đã gắn liền với việc tìm kiếm đường đi tối ưu thông qua cơ chế mô phỏng hành vi của đàn kiến trong tự nhiên. Khi ánh xạ bài toán vào mô hình ACO, mỗi đỉnh của đồ thị được xem như một vị trí mà kiến có thể đi qua, còn các cạnh chính là những con đường nối giữa các vị trí đó.

Trong quá trình thực hiện, mỗi con kiến sẽ bắt đầu từ đỉnh nguồn và di chuyển qua các đỉnh khác để tìm đến đỉnh đích. Tại mỗi bước, việc lựa chọn hướng đi được quyết định dựa trên lượng pheromone tích lũy trên các cạnh và thông tin heuristic, thường liên quan đến độ dài của cạnh. Những con đường ngắn hơn có xu hướng được lựa chọn nhiều hơn do vừa có heuristic tốt, vừa được tăng cường pheromone nhanh hơn qua các vòng lặp.

Một điểm mạnh của ACO là khả năng tìm kiếm trong không gian nghiệm lớn mà không cần duyệt toàn bộ các khả năng. Thay vào đó, thuật toán sử dụng nhiều cá thể hoạt động song song để khám phá các đường đi khác nhau, từ đó dần dần hội tụ về lời giải tốt. Điều này đặc biệt hữu ích khi đồ thị có kích thước lớn hoặc có nhiều đường đi phức tạp mà các phương pháp truyền thống gặp khó khăn.

Ngoài ra, ACO còn có khả năng thích nghi tốt với các biến thể của bài toán, chẳng hạn như khi trọng số của các cạnh thay đổi hoặc khi môi trường có yếu tố động. Nhờ cơ chế cập nhật pheromone liên tục, thuật toán có thể điều chỉnh hướng tìm kiếm theo thời gian, thay vì bị cố định vào một lời giải ban đầu.

Tuy nhiên, việc áp dụng ACO cũng đòi hỏi phải lựa chọn các tham số phù hợp như số lượng kiến, hệ số ảnh hưởng của pheromone và heuristic, hay tốc độ bay hơi. Nếu các tham số này không được thiết lập hợp lý, thuật toán có thể hội tụ chậm hoặc rơi

vào nghiệm cục bộ. Do đó, trong quá trình xây dựng và thực nghiệm, việc tinh chỉnh tham số là một bước quan trọng để đạt được hiệu quả tốt nhất.

1.5. Kết luận chương 1

Trong chương này, báo cáo đã trình bày những cơ sở lý thuyết quan trọng làm nền tảng cho việc xây dựng và triển khai thuật toán trong các chương tiếp theo. Trước hết là tổng quan về tính toán thông minh, một hướng tiếp cận hiện đại giúp giải quyết hiệu quả các bài toán tối ưu phức tạp. Tiếp đó, các khái niệm cơ bản về đồ thị, đồ thị có trọng số và bài toán tìm đường đi ngắn nhất đã được giới thiệu nhằm làm rõ đối tượng nghiên cứu của đề tài.

Đặc biệt, chương đã tập trung trình bày chi tiết về thuật toán Ant Colony Optimization, từ ý tưởng xuất phát trong tự nhiên cho đến các thành phần cốt lõi như pheromone, cơ chế lựa chọn đường đi và quy tắc cập nhật. Thông qua đó, có thể thấy ACO là một phương pháp có nhiều tiềm năng trong việc giải quyết bài toán tìm đường đi ngắn nhất.

Những nội dung lý thuyết được trình bày trong chương này sẽ là cơ sở để nhóm tiến hành xây dựng mô hình thuật toán, thiết lập các công thức và triển khai chương trình trong chương tiếp theo.

CHƯƠNG 2: XÂY DỰNG THUẬT TOÁN

Nếu ở Chương 1, báo cáo đã trình bày cơ sở lý thuyết về bài toán tìm đường đi ngắn nhất trên đồ thị và nguyên lý hoạt động của thuật toán Ant Colony Optimization (ACO), thì trong chương này, các nội dung đó được chuyển hóa thành mô hình thuật toán cụ thể để phục vụ cho việc cài đặt chương trình ở Chương 3. Nói cách khác, Chương 1 đóng vai trò là nền tảng lý thuyết, còn Chương 2 là bước trung gian quan trọng giúp biến các khái niệm mang tính học thuật thành các bước xử lý rõ ràng, có thể hiện thực hóa bằng chương trình máy tính.

Trong phạm vi đề tài, nhóm tập trung xây dựng thuật toán ACO cho bài toán tìm đường đi ngắn nhất giữa một đỉnh nguồn và một đỉnh đích xác định trên đồ thị có trọng số dương. Quá trình xây dựng thuật toán không chỉ dừng lại ở việc nêu lại các công thức cơ bản của ACO, mà còn cần xác định cách biểu diễn dữ liệu đầu vào, cơ chế tạo lời giải của từng cá thể kiến, quy tắc cập nhật pheromone, tiêu chí đánh giá chất lượng lời giải và các tham số điều khiển quá trình hội tụ. Những nội dung này sẽ tạo thành khung thuật toán hoàn chỉnh, làm cơ sở trực tiếp cho phần cài đặt chương trình và thực nghiệm trong chương tiếp theo.

2.1 Phát biểu bài toán

Bài toán được xét trong đề tài là bài toán tìm đường đi ngắn nhất giữa hai đỉnh trên đồ thị có trọng số. Cho một đồ thị $G = (V, E)$, trong đó:

- V là tập đỉnh của đồ thị;
- E là tập cạnh nối giữa các đỉnh;
- mỗi cạnh $(i, j) \in E$ được gán một trọng số $w_{ij} > 0$, biểu diễn chi phí di chuyển từ đỉnh i đến đỉnh j.

Với một đỉnh nguồn $s \in V$ và một đỉnh đích $t \in V$, mục tiêu của bài toán là tìm một đường đi P từ s đến t sao cho tổng trọng số trên đường đi đó là nhỏ nhất.

Giả sử đường đi P có dạng:

$$P = (v_0, v_1, v_2, \dots, v_k)$$

trong đó $v_0 = s$, $v_k = t$, và $(v_r, v_{r+1}) \in E$ với mọi $r = 0,1, \dots, k-1$. Khi đó, độ dài của đường đi được xác định bởi:

$$L(P) = \sum_{r=0}^{k-1} w(v_r, v_{r+1})$$

Bài toán đặt ra là tìm đường đi P^* sao cho:

$$P^* = \arg\min_{P:s\to t} L(P)$$

Trong đề tài này, một số giả thiết được đặt ra để đơn giản hóa quá trình xây dựng mô hình và thuận lợi cho việc cài đặt:

- Đồ thị là đồ thị có trọng số dương.
- Trọng số các cạnh không thay đổi trong quá trình thuật toán chạy.
- Bài toán chỉ xét một cặp nguồn–đích xác định.
- Chương trình ưu tiên xây dựng đường đi không lặp đỉnh nhằm hạn chế chu trình không cần thiết và giảm chi phí tìm kiếm.

Những giả thiết trên phù hợp với mục tiêu của đề tài là mô phỏng và kiểm chứng khả năng áp dụng của ACO vào bài toán tìm đường đi ngắn nhất trên đồ thị, đồng thời tạo cơ sở rõ ràng cho phần thực nghiệm ở Chương 3.

2.2. Mô hình bài toán trên đồ thị có trọng số

Để áp dụng ACO, trước hết cần mô hình hóa dữ liệu đầu vào theo cách phù hợp với cả bài toán lý thuyết lẫn chương trình thực thi. Trong đề tài này, đồ thị được biểu diễn chủ yếu bằng **ma trận trọng số**, vì đây là cách biểu diễn trực quan, thuận tiện cho việc nhập dữ liệu và xử lý trong Python. Đồng thời, từ ma trận trọng số, chương trình có thể suy ra tập đỉnh kề của mỗi đỉnh để phục vụ cho bước tính xác suất lựa chọn đường đi.

Gọi $W = [w_{ij}]$ là ma trận trọng số của đồ thị, trong đó:

$$w_{ij} = \begin{cases} \text{trọng số của cạnh } (i,j), & \text{nếu tồn tại cạnh} \\ 0, & \text{nếu không tồn tại cạnh} \end{cases}$$

Từ ma trận W, ta xác định được tập đỉnh kề của đỉnh i là:

$$N(i) = \{j \in V \mid w_{ij} > 0, j \neq i\}$$

Mỗi đường đi từ nguồn đến đích sẽ được biểu diễn dưới dạng một dãy đỉnh. Đây cũng chính là dạng nghiệm mà mỗi cá thể kiến sẽ xây dựng trong quá trình chạy thuật toán. Như vậy, thay vì duyệt toàn bộ các đường đi có thể có giữa s và t, ACO tiếp cận bài toán theo hướng mô phỏng nhiều cá thể kiến cùng tìm kiếm lời giải, sau đó dùng cơ chế pheromone để ghi nhớ và tăng cường các đường đi tốt.

Từ góc nhìn tối ưu hóa, có thể xem mỗi đường đi từ nguồn đến đích là một nghiệm ứng viên. Tập nghiệm của bài toán thường rất lớn, đặc biệt khi số đỉnh tăng lên hoặc khi giữa các đỉnh tồn tại nhiều nhánh rẽ. Chính vì vậy, việc sử dụng một thuật toán tìm kiếm dựa

trên xác suất như ACO là phù hợp, bởi nó cho phép khám phá không gian nghiệm theo cách linh hoạt, không đòi hỏi phải liệt kê toàn bộ các đường đi có thể có.

2.3. Lựa chọn mô hình ACO cho đề tài

Ant Colony Optimization là một họ thuật toán, trong đó có nhiều biến thể khác nhau như Ant System, Ant Colony System, Max-Min Ant System,... Trong phạm vi đề tài này, nhóm lựa chọn mô hình Ant System cơ bản, vì đây là biến thể đơn giản, dễ hiểu, phù hợp với mục tiêu nghiên cứu và mô phỏng nguyên lý hoạt động của ACO trên bài toán tìm đường đi ngắn nhất.

Lý do của lựa chọn này là:

Thứ nhất, Ant System thể hiện rõ tinh thần của ACO: nhiều cá thể kiến cùng tham gia tìm kiếm, mỗi cá thể xây dựng một lời giải độc lập, sau đó toàn bộ đàn cùng đóng góp vào quá trình cập nhật pheromone. Cách tiếp cận này giúp người đọc dễ hình dung cơ chế "học tập tập thể" của đàn kiến.

Thứ hai, mô hình Ant System đủ đơn giản để hiện thực hóa bằng chương trình Python trong phạm vi một đề tài học phần, nhưng vẫn giữ được đầy đủ các thành phần cốt lõi như khởi tạo pheromone, xác suất chọn cạnh, bay hơi pheromone, tăng cường pheromone và theo dõi lời giải tốt nhất.

Thứ ba, kết quả của Ant System dễ đối chiếu với các thuật toán truyền thống như Dijkstra trong phần thực nghiệm, từ đó giúp đánh giá rõ hơn ưu điểm, hạn chế và tính khả thi của hướng tiếp cận.

Như vậy, trong Chương 2 này, khi nói đến thuật toán ACO, thực chất nhóm đang xây dựng mô hình dựa trên biến thể Ant System cơ bản, sau đó triển khai trong chương trình mô phỏng ở Chương 3.

2.4. Các đại lượng và tham số của thuật toán

Để xây dựng thuật toán, cần xác định rõ các đại lượng và tham số sẽ được sử dụng trong toàn bộ quá trình tìm kiếm. Những đại lượng này không chỉ có ý nghĩa toán học mà còn đóng vai trò là cầu nối trực tiếp giữa mô hình lý thuyết và phần cài đặt chương trình.

2.4.1. Ma trận pheromone

Pheromone được mô hình hóa bởi ma trận:

$$T=[\tau_{ij}]$$

trong đó τ_{ij} biểu diễn lượng pheromone hiện có trên cạnh nối giữa hai đỉnh i và j. Giá trị này phản ánh mức độ hấp dẫn của cạnh đó đối với các cá thể kiến khi lựa chọn đường đi.

Ở thời điểm ban đầu, do chưa có bất kỳ kinh nghiệm nào về chất lượng của các tuyến đường, tất cả các cạnh hợp lệ thường được gán cùng một lượng pheromone khởi tạo:

$$\tau_{ij}(0) = \tau_0$$

với mọi cạnh $(i, j) \in E$.

Việc khởi tạo đồng đều giúp quá trình khám phá ban đầu của thuật toán diễn ra công bằng, tránh hiện tượng thiên lệch quá sớm vào một tuyến đường cụ thể.

2.4.2. Thông tin heuristic

Bên cạnh pheromone, thuật toán còn sử dụng thông tin heuristic để đánh giá "độ tốt cục bộ" của một cạnh tại thời điểm lựa chọn. Trong bài toán tìm đường đi ngắn nhất, heuristic thường được chọn là nghịch đảo của trọng số cạnh:

$$\eta_{ij} = \frac{1}{w_{ij}}$$

nếu cạnh (i, j) tồn tại. Khi đó, cạnh có trọng số càng nhỏ thì giá trị heuristic càng lớn, đồng nghĩa với việc cạnh đó càng có xu hướng được ưu tiên.

Thông tin heuristic đóng vai trò đặc biệt quan trọng trong giai đoạn đầu của thuật toán, khi pheromone chưa có sự khác biệt rõ ràng giữa các cạnh. Nhờ heuristic, đàn kiến vẫn có thể định hướng tìm kiếm về phía các cạnh ngắn hơn, thay vì di chuyển hoàn toàn ngẫu nhiên.

2.4.3. Tập đỉnh có thể lựa chọn

Khi một cá thể kiến đang đứng tại đỉnh i, nó không thể chọn tùy ý bất kỳ đỉnh nào trên đồ thị, mà chỉ được chọn trong tập đỉnh kề hợp lệ. Với kiến thứ k, gọi N_i^k là tập các đỉnh mà kiến có thể đi tới từ đỉnh i ở bước hiện tại.

Trong đề tài này, để hạn chế hiện tượng tạo vòng lặp không cần thiết, chương trình loại bỏ những đỉnh mà cá thể kiến đã đi qua trước đó. Như vậy:

$$N_i^k = \{j \in N(i) \mid j \notin \text{tabu}_k\}$$

trong đó $tabu_k$là danh sách hoặc tập các đỉnh mà kiến thứ kđã đi qua. Cơ chế này giúp mỗi cá thể kiến ưu tiên xây dựng đường đi đơn giản, đồng thời làm giảm khả năng mắc kẹt trong các chu trình.

2.4.4. Các tham điều khiển

Thuật toán sử dụng một số tham số chính sau:

- m: số lượng kiến trong mỗi vòng lặp;
- N: số vòng lặp tối đa;
- α: hệ số ảnh hưởng của pheromone;
- β: hệ số ảnh hưởng của heuristic;
- ρ: hệ số bay hơi pheromone;
- Q: hằng số dùng để tính lượng pheromone bổ sung;
- τ_0: giá trị pheromone khởi tạo.

Mỗi tham số đều tác động đến hành vi tìm kiếm của thuật toán. Nếu chọn không phù hợp, thuật toán có thể hội tụ quá sớm, hội tụ chậm hoặc cho kết quả không ổn định. Vì vậy, việc lựa chọn tham số là một phần quan trọng trong xây dựng mô hình.

2.5. Xây dựng lời giải bằng hành vi của từng cá thể kiến

Trọng tâm của thuật toán ACO nằm ở quá trình mỗi cá thể kiến tự xây dựng một đường đi từ đỉnh nguồn đến đỉnh đích. Quá trình này được thực hiện theo từng bước, tại mỗi bước kiến sẽ lựa chọn đỉnh tiếp theo dựa trên xác suất. Nhờ cách làm này, lời giải được hình thành dần dần thay vì được tính toán trực tiếp như ở các thuật toán tất định.

Giả sử tại một thời điểm nào đó, kiến thứ kđang đứng tại đỉnh i. Với mỗi đỉnh $j$thuộc tập lựa chọn N_i^k, xác suất để kiến chọn đi đến jđược xác định bởi công thức:

$$P_{ij}^k = \frac{[\tau_{il}]^\alpha [\eta_{il}]^\beta}{\sum_{l \in N_i^k} [\tau_{il}]^\alpha [\eta_{il}]^\beta}$$

Trong công thức trên, hai thành phần quan trọng nhất là pheromone và heuristic. Nếu α lớn, kiến sẽ bị ảnh hưởng mạnh bởi kinh nghiệm tích lũy từ các vòng lặp trước; nếu β lớn, kiến sẽ ưu tiên hơn cho các cạnh ngắn ở thời điểm hiện tại. Sự kết hợp giữa hai yếu tố này tạo nên sự cân bằng giữa **khai thác** những đường đi đã được đánh giá tốt và **khám phá** những lựa chọn mới.

Sau khi tính được xác suất cho tất cả các đỉnh ứng viên, chương trình sử dụng cơ chế chọn ngẫu nhiên theo phân phối xác suất, thường được cài đặt bằng phương pháp "roulette wheel". Nhờ đó, một cạnh có xác suất cao sẽ có nhiều khả năng được chọn

hơn, nhưng các cạnh khác vẫn không bị loại bỏ hoàn toàn. Đây chính là điểm khác biệt quan trọng của ACO so với các thuật toán tham lam.

Quá trình trên được lặp lại cho đến khi xảy ra một trong hai trường hợp:

- kiến đi tới được đỉnh đích t, khi đó thu được một lời giải hợp lệ;
- kiến không còn đỉnh hợp lệ nào để tiếp tục trước khi đến đích, khi đó lời giải được xem là không hợp lệ và sẽ không tham gia vào bước tăng cường pheromone.

Việc cho phép tồn tại các lời giải không hợp lệ phản ánh đúng bản chất khám phá của ACO. Tuy nhiên, để tránh ảnh hưởng tiêu cực đến chất lượng hội tụ, chỉ những đường đi hợp lệ mới được dùng để cập nhật pheromone trong đề tài này.

2.6. Đánh giá chất lượng lời giải

Sau khi mỗi cá thể kiến hoàn thành hành trình của mình, chương trình cần đánh giá chất lượng của các đường đi thu được. Đối với bài toán đang xét, tiêu chí đánh giá là tổng trọng số của đường đi.

Với đường đi của kiến thứ k:

$$P^k = (v_0, v_1, v_2, \dots, v_h)$$

trong đó $v_0 = s$ và $v_h = t$, độ dài đường đi được tính bởi:

$$L_k = \sum_{r=0}^{h-1} w(v_r, v_{r+1})$$

Đường đi nào có giá trị L_k nhỏ hơn thì được xem là tốt hơn. Trong mỗi vòng lặp, chương trình sẽ xác định:

- lời giải tốt nhất của vòng lặp hiện tại;
- lời giải tốt nhất toàn cục từ đầu đến thời điểm hiện tại.

Việc duy trì hai mức đánh giá này mang lại nhiều lợi ích. Lời giải tốt nhất của từng vòng lặp giúp phản ánh chất lượng tìm kiếm trong phạm vi cục bộ, còn lời giải tốt nhất toàn cục là kết quả cuối cùng mà thuật toán hướng đến. Đồng thời, giá trị tốt nhất toàn cục cũng được lưu lại sau mỗi vòng lặp để xây dựng biểu đồ hội tụ ở Chương 3.

Có thể thấy, bước đánh giá lời giải là mắt xích kết nối giữa quá trình mô phỏng sự di chuyển của kiến và bước cập nhật pheromone. Nếu không có cơ chế đo lường chất lượng lời giải, thuật toán sẽ không thể phân biệt được đâu là đường đi tốt để tăng cường, đâu là đường đi kém để dần bị loại bỏ.

2.7. Cơ chế cập nhật pheromone

Sau khi toàn bộ tập kiến hoàn thành quá trình xây dựng lời giải trong một vòng lặp, ma trận pheromone sẽ được cập nhật. Đây là bước thể hiện rõ nhất cơ chế học tập tập thể của đàn kiến.

Quá trình cập nhật pheromone gồm hai thành phần: bay hơi và bổ sung pheromone.

2.7.1. Bay hơi pheromone

Trước hết, pheromone trên tất cả các cạnh được giảm đi một phần theo hệ số ρ:

$$\tau_{ij}(t+1) = (1-\rho)\tau_{ij}(t)$$

nếu chỉ xét riêng bước bay hơi.

Bay hơi pheromone có vai trò rất quan trọng. Nếu không có cơ chế này, những cạnh từng được chọn nhiều ở giai đoạn đầu sẽ luôn duy trì ưu thế tuyệt đối, khiến thuật toán dễ bị "kẹt" trong một lời giải cục bộ. Nhờ bay hơi, ảnh hưởng của các quyết định cũ sẽ giảm dần theo thời gian, tạo điều kiện cho thuật toán tiếp tục khám phá các khả năng khác.

2.7.2. Bổ sung pheromone

Sau bước bay hơi, các đường đi hợp lệ mà kiến vừa tìm được sẽ đóng góp thêm pheromone lên những cạnh mà chúng đã đi qua. Lượng pheromone bổ sung trên cạnh (i, j) được xác định bởi:

$$\tau_{ij}(t+1) = (1-\rho)\tau_{ij}(t) + \Delta\tau_{ij}$$

trong đó:

$$\Delta\tau_{ij} = \sum_{k=1}^{m} \Delta\tau_{ij}^{k}$$

và

$$\Delta\tau_{ij}^{k} = \begin{cases} \dfrac{Q}{L_k}, & \text{nếu kiến } k \text{ đi qua cạnh } (i,j) \\ 0, & \text{ngược lại} \end{cases}$$

Công thức trên cho thấy một đường đi càng ngắn thì lượng pheromone nó để lại càng lớn. Nhờ vậy, các cạnh thuộc những đường đi tốt sẽ ngày càng hấp dẫn hơn trong các vòng lặp tiếp theo. Đây là cơ chế giúp thuật toán dần hội tụ về lời giải tối ưu hoặc gần tối ưu.

Trong đề tài này, nhóm sử dụng cách cập nhật dựa trên **toàn bộ các kiến hợp lệ** thay vì chỉ cập nhật theo kiến tốt nhất. Cách làm này phù hợp với mô hình Ant System cơ bản, đồng thời giữ được tính tập thể của đàn kiến. Trên các đồ thị có kích thước nhỏ và trung bình, phương pháp này vừa đơn giản trong cài đặt, vừa đủ hiệu quả để quan sát xu hướng hội tụ.

2.7.3. Ý nghĩa của cơ chế cập nhật

Cơ chế cập nhật pheromone là nhân tố quyết định khả năng học của thuật toán. Nếu coi mỗi vòng lặp là một lần thử, thì pheromone chính là "dấu vết kinh nghiệm" được lưu lại trên đồ thị. Qua nhiều vòng lặp, các cạnh tốt được củng cố lặp đi lặp lại, trong khi các cạnh kém hiệu quả dần mất sức hút do bay hơi.

Có thể nói, trong thuật toán ACO, lời giải tối ưu không xuất hiện ngay lập tức từ một phép tính trực tiếp, mà được hình thành dần thông qua sự tương tác giữa nhiều cá thể đơn giản. Chính điều này tạo nên giá trị đặc trưng của ACO so với các thuật toán tìm đường đi truyền thống.

2.8. Quy trình thực hiện của thuật toán

Từ các thành phần đã trình bày, có thể mô tả toàn bộ quy trình hoạt động của thuật toán như sau.

Ban đầu, chương trình đọc dữ liệu đồ thị, xác định đỉnh nguồn, đỉnh đích và các tham số điều khiển. Sau đó, ma trận pheromone được khởi tạo đồng đều, đồng thời ma trận heuristic được tính từ ma trận trọng số của đồ thị.

Ở mỗi vòng lặp, lần lượt từng cá thể kiến sẽ xuất phát từ đỉnh nguồn, lựa chọn đỉnh tiếp theo theo công thức xác suất và xây dựng đường đi cho đến khi tới được đỉnh đích hoặc không còn khả năng mở rộng. Sau khi toàn bộ đàn kiến hoàn thành hành trình, chương trình tính độ dài các đường đi hợp lệ, cập nhật lời giải tốt nhất của vòng lặp và lời giải tốt nhất toàn cục.

Tiếp theo, thuật toán thực hiện bay hơi pheromone trên toàn bộ các cạnh, rồi cộng thêm pheromone mới từ các đường đi mà đàn kiến vừa xây dựng. Giá trị lời giải tốt nhất toàn cục được lưu lại để theo dõi xu hướng hội tụ.

Quy trình này được lặp lại cho đến khi đạt số vòng lặp tối đa hoặc thỏa mãn điều kiện dừng đã đặt ra. Cuối cùng, thuật toán trả về đường đi tốt nhất cùng tổng chi phí tương ứng.

Để làm rõ hơn, có thể mô tả thuật toán tổng quát dưới dạng giả mã như sau.

Thuật toán 2.1. Giả mã ACO cho bài toán tìm đường đi ngắn nhất

Input: Đồ thị G=(V,E), ma trận trọng số W, đỉnh nguồn s, đỉnh đích t
Các tham số m, N, alpha, beta, rho, Q, tau0
Output: Đường đi tốt nhất best_path, độ dài nhỏ nhất best_length

1. Khởi tạo pheromone tau_ij = tau0 với mọi cạnh (i,j) thuộc E
2. Tính heuristic eta_ij = 1 / w_ij nếu tồn tại cạnh (i,j)
3. best_path = rỗng, best_length = vô cùng

4. Với iter = 1 đến N:
5. Tập lời giải của vòng lặp = rỗng
6. Với mỗi kiến k = 1 đến m:
7. current = s
8. path = [s]
9. tabu = {s}
10. Trong khi current != t:
11. Xác định tập ứng viên N_current^k
12. Nếu tập ứng viên rỗng:
13. path không hợp lệ, dừng kiến k
14. Tính xác suất chọn từng đỉnh ứng viên
15. Chọn đỉnh tiếp theo theo roulette wheel
16. Cập nhật path, tabu, current
17. Nếu path hợp lệ:
18. Tính độ dài Lk của path
19. Lưu (path, Lk) vào tập lời giải của vòng lặp
20. Nếu Lk < best_length:
21. best_length = Lk
22. best_path = path
23. Bay hơi pheromone trên tất cả các cạnh
24. Với mỗi lời giải hợp lệ (path, Lk):
25. Với mỗi cạnh (i,j) thuộc path:
26. tau_ij = tau_ij + Q / Lk
27. Lưu best_length vào lịch sử hội tụ
28. Trả về best_path, best_length

Giả mã trên cho thấy cấu trúc của thuật toán khá rõ ràng và có tính mô-đun cao. Đây cũng chính là cơ sở để chương trình trong Chương 3 được tổ chức thành các khối chức năng như nhập dữ liệu, khởi tạo pheromone, mô phỏng đàn kiến, cập nhật pheromone, lưu kết quả và trực quan hóa.

2.9. Phân tích vai trò của các tham số

Một trong những vấn đề quan trọng khi xây dựng thuật toán ACO là lựa chọn tham số. Dù mô hình lý thuyết của ACO khá rõ ràng, hiệu quả thực tế của thuật toán lại phụ thuộc nhiều vào cách thiết lập các giá trị điều khiển.

2.9.1. Ảnh hưởng của α

Tham số α quyết định mức độ ảnh hưởng của pheromone đến xác suất lựa chọn đường đi. Khi α lớn, đàn kiến sẽ có xu hướng bám sát các cạnh đã được tích lũy nhiều pheromone, nghĩa là thuật toán tập trung mạnh vào việc khai thác kinh nghiệm đã có. Điều này có thể giúp hội tụ nhanh hơn, nhưng cũng dễ dẫn đến hội tụ sớm vào nghiệm cục bộ.

Ngược lại, nếu α quá nhỏ, vai trò của pheromone trở nên mờ nhạt, khiến thuật toán khó tận dụng được thông tin tích lũy qua các vòng lặp.

2.9.2. Ảnh hưởng của β

Tham số β thể hiện mức độ ưu tiên của thông tin heuristic, tức là mức độ kiến có xu hướng chọn các cạnh ngắn hơn ngay tại thời điểm ra quyết định. Với bài toán đường đi ngắn nhất, heuristic đóng vai trò quan trọng, vì bản thân trọng số cạnh đã phản ánh trực tiếp chi phí di chuyển.

Trong thực nghiệm của đề tài, việc chọn β lớn hơn α là hợp lý, bởi nó giúp đàn kiến sớm ưu tiên các cạnh có chi phí nhỏ, đồng thời vẫn giữ được tác dụng dẫn hướng của pheromone qua thời gian.

2.9.3. Ảnh hưởng của ρ

Hệ số ρ điều khiển tốc độ bay hơi pheromone. Nếu ρ quá nhỏ, pheromone tồn tại quá lâu trên đồ thị, khiến thuật toán có thể bị chi phối mạnh bởi những lựa chọn ban đầu. Ngược lại, nếu ρ quá lớn, pheromone bị mất quá nhanh, làm suy yếu khả năng học tập của đàn kiến.

Do đó, ρ cần được chọn sao cho thuật toán vừa ghi nhớ được kinh nghiệm tốt, vừa đủ linh hoạt để tránh rơi vào các đường đi chưa tối ưu.

2.9.4. Ảnh hưởng của số lượng kiến và số vòng lặp

Số lượng kiến m và số vòng lặp N quyết định trực tiếp quy mô tìm kiếm của thuật toán. Nếu số kiến quá ít hoặc số vòng lặp quá nhỏ, không gian nghiệm sẽ không được khám phá đầy đủ. Ngược lại, nếu tăng quá lớn, chất lượng lời giải có thể cải thiện nhưng thời gian thực thi cũng tăng lên đáng kể.

Trong đề tài, các tham số này được chọn ở mức vừa phải để cân bằng giữa chất lượng lời giải và chi phí tính toán, phù hợp với mục tiêu mô phỏng trên các đồ thị có kích thước nhỏ đến trung bình.

2.9.5. Liên hệ với phần thực nghiệm

Các tham số được sử dụng trong Chương 3 như số lượng kiến, số vòng lặp, α, β, ρ, Q và τ_0 thực chất chính là sự cụ thể hóa của mô hình đã xây dựng trong chương này. Nhờ đó, phần cài đặt không chỉ đơn thuần là viết mã, mà là quá trình hiện thực hóa trực tiếp các công thức và quy tắc đã được xác lập về mặt thuật toán.

2.10. Phân tích độ phức tạp và đặc điểm của thuật toán

Bên cạnh việc xây dựng mô hình, cần xem xét sơ bộ độ phức tạp của thuật toán để hiểu rõ hơn về hiệu quả tính toán của phương pháp.

Giả sử đồ thị có n đỉnh. Trong một vòng lặp, mỗi cá thể kiến có thể đi qua tối đa khoảng n đỉnh. Ở mỗi bước, kiến cần xem xét tập đỉnh kề và tính xác suất lựa chọn. Nếu biểu diễn bằng ma trận trọng số, việc xác định tập ứng viên có thể tốn đến $O(n)$ phép kiểm tra ở mỗi bước. Khi đó, chi phí xây dựng lời giải của một kiến có thể đạt cỡ $O(n^2)$ trong trường hợp xấu.

Với m kiến và N vòng lặp, độ phức tạp tổng quát của thuật toán có thể xem xấp xỉ:

$$O(N \cdot m \cdot n^2)$$

chưa kể đến chi phí cập nhật pheromone và các thao tác phụ trợ khác. Bộ nhớ cần thiết chủ yếu dùng để lưu ma trận trọng số, ma trận pheromone và các cấu trúc lưu đường đi, nên thường vào khoảng:

$$O(n^2)$$

với cách biểu diễn bằng ma trận.

Phân tích trên cho thấy ACO có chi phí tính toán cao hơn so với các thuật toán tất định như Dijkstra khi giải bài toán đường đi ngắn nhất trên đồ thị nhỏ và tĩnh. Tuy nhiên,

điểm mạnh của ACO không nằm ở tốc độ trên các bài toán cơ bản, mà ở khả năng mở rộng sang các bài toán tối ưu tổ hợp phức tạp, nơi không gian nghiệm lớn và lời giải tối ưu khó thu được bằng các phương pháp truyền thống.

Chính vì vậy, trong đề tài này, ACO không chỉ được xem như một công cụ để tìm đường đi ngắn nhất, mà còn là một mô hình tiêu biểu cho hướng tiếp cận tính toán thông minh, có khả năng phát triển thêm trong các nghiên cứu sau.

2.11. Kết luận chương 2

Trong chương này, báo cáo đã chuyển từ phần cơ sở lý thuyết sang bước xây dựng mô hình thuật toán cụ thể cho bài toán tìm đường đi ngắn nhất trên đồ thị bằng phương pháp Ant Colony Optimization. Trước hết, bài toán đã được phát biểu dưới dạng hình thức với đầu vào là đồ thị có trọng số dương, đỉnh nguồn và đỉnh đích; đầu ra là đường đi có tổng chi phí nhỏ nhất. Trên cơ sở đó, báo cáo đã tiến hành mô hình hóa dữ liệu đầu vào bằng ma trận trọng số, xác định các đại lượng cần thiết như pheromone, heuristic và tập đỉnh ứng viên cho từng cá thể kiến.

Tiếp theo, chương đã trình bày cách xây dựng lời giải của từng cá thể kiến thông qua cơ chế lựa chọn xác suất chịu ảnh hưởng bởi pheromone và thông tin heuristic. Đồng thời, cơ chế đánh giá đường đi, cập nhật pheromone, vai trò của các tham số điều khiển và quy trình tổng quát của thuật toán cũng đã được phân tích rõ ràng. Những nội dung này không chỉ làm rõ nguyên lý vận hành của mô hình ACO được sử dụng trong đề tài, mà còn tạo thành nền tảng trực tiếp cho phần hiện thực hóa chương trình.

Có thể nói, Chương 2 giữ vai trò trung gian quan trọng giữa phần lý thuyết ở Chương 1 và phần cài đặt, thực nghiệm ở Chương 3. Nếu Chương 1 giúp làm rõ vì sao ACO có thể được áp dụng cho bài toán đường đi ngắn nhất, thì Chương 2 trả lời câu hỏi thuật toán đó được xây dựng như thế nào để có thể cài đặt thành chương trình cụ thể. Trên cơ sở mô hình đã thiết lập ở đây, Chương 3 sẽ trình bày quá trình hiện thực hóa thuật toán bằng Python, tổ chức cấu trúc chương trình, chạy thử trên dữ liệu mẫu và đánh giá kết quả thực nghiệm.

CHƯƠNG 3: CÀI ĐẶT CHƯƠNG TRÌNH VÀ THỰC NGHIỆM

Chương này trình bày quá trình cài đặt chương trình mô phỏng thuật toán Ant Colony Optimization (ACO) cho bài toán tìm đường đi ngắn nhất trên đồ thị bằng ngôn ngữ Python. Nếu ở các chương trước, báo cáo đã tập trung làm rõ cơ sở lý thuyết của bài toán, nguyên lý hoạt động của thuật toán và mô hình xây dựng lời giải, thì trong chương này, các nội dung đó được hiện thực hóa thành một chương trình cụ thể nhằm kiểm chứng khả năng áp dụng của thuật toán trong thực tế.

Việc cài đặt chương trình không chỉ giúp mô phỏng lại quá trình các cá thể kiến di chuyển trên đồ thị và cộng tác thông qua cơ chế pheromone, mà còn tạo cơ sở để quan sát, đánh giá và phân tích kết quả tìm kiếm sau nhiều vòng lặp. Thông qua quá trình thực nghiệm, nhóm có thể kiểm tra mức độ hội tụ của thuật toán, khả năng tìm được đường đi tốt, cũng như xem xét ảnh hưởng của các tham số đến hiệu quả hoạt động của chương trình. Đây là bước chuyển quan trọng từ phần nghiên cứu lý thuyết sang phần kiểm chứng thực nghiệm của đề tài.

3.1. Môi trường cài đặt

Chương trình được cài đặt bằng ngôn ngữ Python, một ngôn ngữ lập trình bậc cao được sử dụng rộng rãi trong các lĩnh vực khoa học dữ liệu, tối ưu hóa và trí tuệ nhân tạo. Việc lựa chọn Python xuất phát từ nhiều ưu điểm như cú pháp rõ ràng, dễ đọc, dễ triển khai thuật toán và có hệ sinh thái thư viện phong phú hỗ trợ tốt cho việc xử lý dữ liệu, mô phỏng và trực quan hóa kết quả. Điều này đặc biệt phù hợp với yêu cầu của đề tài, vốn cần vừa xây dựng thuật toán, vừa biểu diễn quá trình thực nghiệm một cách trực quan.

Trong quá trình thực hiện, chương trình được tổ chức theo hướng mô-đun nhằm thuận tiện cho việc xây dựng, kiểm thử và mở rộng. Các thành phần chính của hệ thống được tách thành các phần chức năng riêng như xử lý dữ liệu đồ thị, mô phỏng sự di chuyển của kiến, cập nhật pheromone, tìm đường đi tốt nhất và trực quan hóa kết quả. Cách tổ chức này giúp mã nguồn rõ ràng hơn, dễ theo dõi luồng xử lý và thuận lợi cho việc điều chỉnh thuật toán khi thay đổi tham số hoặc mở rộng bài toán trong các nghiên cứu tiếp theo.

Về môi trường phát triển, chương trình có thể được triển khai trên các hệ điều hành thông dụng như Windows, Linux hoặc macOS, miễn là máy tính đã cài đặt Python và các thư viện cần thiết. Trong đề tài này, môi trường cài đặt được lựa chọn theo hướng gọn nhẹ, dễ tái sử dụng và phù hợp với mục tiêu mô phỏng thuật toán trên các bộ dữ liệu thử nghiệm có kích thước nhỏ và trung bình. Nhờ đó, việc chạy chương trình, kiểm tra kết quả và đánh giá hiệu quả của thuật toán được thực hiện một cách thuận tiện.

Nhìn chung, môi trường cài đặt được lựa chọn không chỉ đáp ứng tốt yêu cầu kỹ thuật của đề tài mà còn tạo điều kiện thuận lợi cho việc trình bày kết quả thực nghiệm. Đây là cơ sở quan trọng để nhóm triển khai các chức năng chính của chương trình và tiến hành đánh giá thuật toán trong các phần tiếp theo.

3.2. Công cụ và thư viện sử dụng

Để xây dựng chương trình mô phỏng thuật toán ACO, nhóm sử dụng một số công cụ và thư viện hỗ trợ phù hợp với đặc thù của bài toán. Các công cụ này đóng vai trò quan trọng trong việc triển khai mã nguồn, quản lý cấu trúc chương trình, xử lý dữ liệu đồ thị và trực quan hóa kết quả thực nghiệm. Việc lựa chọn đúng công cụ không chỉ giúp rút ngắn thời gian cài đặt mà còn nâng cao tính rõ ràng, chính xác và khả năng mở rộng của chương trình.

Trước hết, Visual Studio Code được sử dụng làm môi trường soạn thảo và phát triển mã nguồn. Đây là một công cụ phổ biến, có giao diện thân thiện và hỗ trợ tốt cho ngôn ngữ Python thông qua các tiện ích mở rộng như gợi ý cú pháp, kiểm tra lỗi và chạy chương trình trực tiếp. Bên cạnh đó, nhóm cũng sử dụng GitHub để lưu trữ và quản lý mã nguồn của project. Việc sử dụng nền tảng này giúp theo dõi cấu trúc chương trình, lưu lại các thay đổi trong quá trình phát triển và thuận tiện cho việc chia sẻ, chỉnh sửa hoặc mở rộng mã nguồn về sau.

Về mặt thư viện lập trình, chương trình sử dụng NumPy để hỗ trợ các phép toán số học và thao tác với dữ liệu dạng mảng, đặc biệt hữu ích trong quá trình biểu diễn và xử lý ma trận trọng số của đồ thị. Thư viện NetworkX được sử dụng để mô hình hóa đồ thị, quản lý tập đỉnh, cạnh và trọng số, đồng thời hỗ trợ biểu diễn cấu trúc đồ thị một cách trực quan và thuận tiện. Ngoài ra, Matplotlib được sử dụng để trực quan hóa kết quả thực nghiệm, chẳng hạn như biểu diễn đồ thị đầu vào, đường đi tốt nhất tìm được và xu hướng hội tụ của thuật toán qua các vòng lặp. Những thư viện này kết hợp với nhau tạo thành nền tảng kỹ thuật phù hợp để hiện thực hóa đầy đủ các yêu cầu của đề tài.

Ngoài các thư viện trên, chương trình còn sử dụng một số thư viện chuẩn có sẵn trong Python nhằm hỗ trợ những thao tác cơ bản như đo thời gian thực thi, xử lý đường dẫn tệp và tổ chức dữ liệu đầu vào, đầu ra. Sự kết hợp giữa các công cụ phát triển và các thư viện hỗ trợ đã giúp chương trình được xây dựng theo hướng rõ ràng, dễ bảo trì và có khả năng phục vụ tốt cho phần thực nghiệm, đánh giá kết quả ở các mục tiếp theo.

3.3. Cấu trúc chương trình

Để thuận tiện cho việc cài đặt, kiểm thử và mở rộng trong quá trình thực hiện đề tài, chương trình được tổ chức theo hướng mô-đun. Thay vì viết toàn bộ mã nguồn trong một tệp duy nhất, các thành phần chức năng được tách riêng thành từng phần tương ứng với các nhiệm vụ chính như xử lý dữ liệu đồ thị, mô phỏng quá trình di chuyển của kiến, thực hiện cập nhật pheromone và trực quan hóa kết quả. Cách tổ chức này giúp chương trình rõ ràng hơn về mặt cấu trúc, dễ theo dõi luồng xử lý và thuận lợi cho việc hiệu chỉnh thuật toán khi thay đổi tham số hoặc thử nghiệm trên các bộ dữ liệu khác nhau.

Về tổng thể, chương trình được xây dựng xoay quanh ba nhóm thành phần chính. Nhóm thứ nhất là phần tiếp nhận và biểu diễn dữ liệu đầu vào, có nhiệm vụ đọc thông tin đồ thị, xác định tập đỉnh, tập cạnh và các trọng số tương ứng để làm cơ sở cho toàn bộ quá trình tính toán. Nhóm thứ hai là phần cài đặt thuật toán ACO, bao gồm các bước khởi tạo pheromone, mô phỏng hành vi tìm đường của các cá thể kiến, tính toán xác suất lựa chọn đỉnh tiếp theo và cập nhật pheromone sau mỗi vòng lặp. Nhóm thứ ba là phần xử lý kết quả, có nhiệm vụ lưu trữ đường đi tốt nhất, thống kê giá trị chi phí tương ứng và hỗ trợ biểu diễn kết quả bằng hình ảnh hoặc biểu đồ trực quan.

Việc tổ chức chương trình theo cấu trúc như trên mang lại nhiều lợi ích trong quá trình phát triển đề tài. Trước hết, mỗi thành phần thực hiện một chức năng tương đối độc lập nên dễ dàng kiểm tra và sửa lỗi khi cần thiết. Bên cạnh đó, cách chia mô-đun còn giúp chương trình có khả năng tái sử dụng tốt hơn, chẳng hạn như có thể thay đổi dữ liệu đầu vào hoặc bổ sung các cơ chế lựa chọn đường đi khác mà không ảnh hưởng quá nhiều đến toàn bộ hệ thống. Đây cũng là cơ sở quan trọng để nhóm triển khai các chức năng chính của thuật toán một cách logic và có hệ thống trong các mục tiếp theo.

```
aco_shortest_path/
├── main.py
├── requirements.txt
├── README.md
├── data/
│   ├── sample_matrix.csv
│   └── sample_graph.txt
├── results/
│   ├── best_path.txt
│   ├── comparison_chart.png
│   ├── comparison_report.txt
│   ├── convergence.png
│   └── graph_result.png
├── src/
│   ├── aco.py
│   ├── ant.py
│   ├── config.py
│   ├── graph_utils.py
│   └── visualize.py
└── tests/
    └── test_small_graph.py
```

Hình 3.1. Cấu trúc tổng quát của chương trình mô phỏng thuật toán ACO

Bảng 3.1. Mô tả ngắn các thành phần chính của chương trình

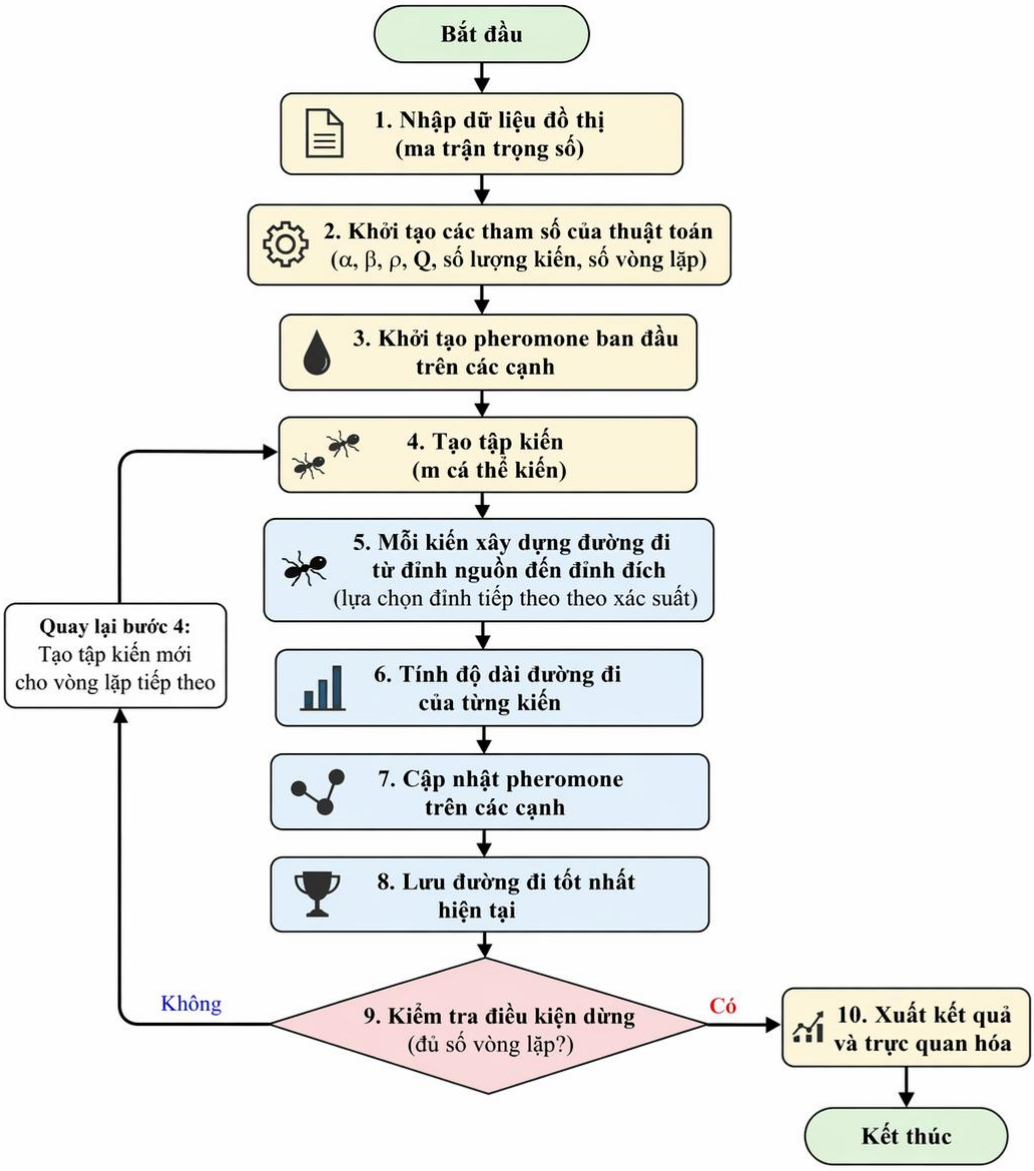
Thành phần	**Nội dung**	**Chức năng chính**
Tệp chương trình chính	main.py	Điều phối toàn bộ quá trình chạy thuật toán
Thư mục dữ liệu	data/	Lưu dữ liệu đồ thị đầu vào dùng cho thử nghiệm
Thư mục kết quả	results/	Lưu đường đi tốt nhất, báo cáo và hình ảnh kết quả
Thư mục mã nguồn	src/	Cài đặt các thành phần chính của thuật toán ACO
Thư mục kiểm thử	tests/	Kiểm tra tính đúng đắn của chương trình trên dữ liệu nhỏ
Tệp phụ trợ	README.md, requirements.txt	Mô tả project và khai báo các thư viện cần cài đặt

3.4. Cài đặt các chức năng chính

Sau khi xây dựng được cấu trúc tổng thể của chương trình, bước tiếp theo là triển khai các chức năng cốt lõi của thuật toán Ant Colony Optimization cho bài toán tìm đường đi ngắn nhất trên đồ thị. Các chức năng này phản ánh trực tiếp quá trình hoạt động của thuật toán, từ khâu tiếp nhận dữ liệu đầu vào cho đến khâu tạo lời giải, đánh giá lời giải và cập nhật thông tin pheromone để phục vụ cho các vòng lặp tiếp theo.

Trong phạm vi đề tài, chương trình được cài đặt theo đúng tinh thần của mô hình ACO cơ bản. Trước hết, đồ thị đầu vào cần được đọc và biểu diễn dưới dạng phù hợp để chương trình có thể khai thác thông tin về các đỉnh, cạnh và trọng số. Tiếp theo, hệ thống tiến hành khởi tạo lượng pheromone ban đầu trên các cạnh, tạo điều kiện để các cá thể kiến bắt đầu quá trình tìm kiếm. Sau đó, các con kiến được mô phỏng di chuyển từ đỉnh nguồn đến đỉnh đích dựa trên xác suất lựa chọn chịu ảnh hưởng bởi pheromone và thông tin heuristic. Kết thúc mỗi vòng lặp, chương trình thực hiện cập nhật pheromone để tăng cường các đường đi tốt và làm mờ dần ảnh hưởng của các đường đi kém hiệu quả.

Việc cài đặt theo từng chức năng riêng biệt không chỉ giúp quá trình hiện thực hóa thuật toán trở nên rõ ràng hơn mà còn tạo thuận lợi cho việc kiểm thử từng phần. Thông qua cách trình bày này, báo cáo có thể mô tả cụ thể vai trò của từng bước trong chương trình, đồng thời làm rõ mối liên hệ giữa mô hình lý thuyết đã trình bày ở Chương 2 và quá trình hiện thực trong môi trường Python.



Hình 3.2 Sơ đồ các chức năng chính của chương trình

3.4.1. Nhập dữ liệu đồ thị

Chức năng đầu tiên của chương trình là tiếp nhận dữ liệu đồ thị từ tệp đầu vào để phục vụ cho quá trình mô phỏng thuật toán. Trong project, chương trình hỗ trợ hai dạng dữ liệu mẫu, gồm ma trận trọng số và danh sách cạnh. Tuy nhiên, ở phiên bản thực nghiệm hiện tại, chương trình chính sử dụng dữ liệu từ tệp ma trận trọng số để xây dựng đồ thị đầu vào.

Sau khi đọc dữ liệu từ tệp, chương trình tiến hành chuyển đổi thông tin này sang dạng biểu diễn phù hợp để phục vụ cho việc tính toán. Từ cấu trúc dữ liệu thu được, hệ thống có thể xác định tập đỉnh, các đỉnh kề và trọng số tương ứng của từng cạnh. Đây

là cơ sở để thuật toán thực hiện các bước tiếp theo như khởi tạo pheromone, tính thông tin heuristic và mô phỏng quá trình di chuyển của các cá thể kiến trên đồ thị.

Việc tách riêng chức năng nhập dữ liệu giúp chương trình trở nên linh hoạt hơn trong quá trình mở rộng, bởi có thể thay đổi hoặc bổ sung định dạng đầu vào mà không ảnh hưởng nhiều đến các thành phần xử lý còn lại. Đồng thời, bước này cũng góp phần bảo đảm tính đúng đắn của chương trình trước khi bắt đầu các vòng lặp tìm kiếm lời giải

3.4.2. Khởi tạo pheromone

Sau khi dữ liệu đồ thị đã được nạp vào chương trình, bước tiếp theo là khởi tạo lượng pheromone ban đầu trên các cạnh. Đây là bước cần thiết để thuật toán ACO có thể bắt đầu quá trình tìm kiếm lời giải. Ở thời điểm ban đầu, thuật toán chưa có bất kỳ kinh nghiệm nào về chất lượng của các tuyến đường, do đó lượng pheromone trên các cạnh thường được gán cùng một giá trị khởi tạo nhằm bảo đảm tính công bằng trong giai đoạn khám phá ban đầu.

Việc khởi tạo pheromone đồng đều giúp các cá thể kiến không bị thiên lệch quá sớm vào một tuyến đường cụ thể khi chưa có đủ thông tin đánh giá. Trong giai đoạn đầu, quá trình lựa chọn hướng đi của kiến sẽ chịu ảnh hưởng kết hợp giữa lượng pheromone ban đầu và thông tin heuristic của cạnh, từ đó cho phép thuật toán khám phá nhiều hướng đi khác nhau trên đồ thị. Cơ chế này góp phần duy trì sự cân bằng giữa khả năng khám phá không gian lời giải và khả năng hội tụ về các đường đi tốt.

Ngoài việc gán giá trị khởi tạo cho các cạnh, chương trình còn cần lưu trữ ma trận pheromone để phục vụ cho quá trình cập nhật ở các vòng lặp sau. Trong suốt quá trình thực hiện thuật toán, lượng pheromone này sẽ liên tục thay đổi dưới tác động của cơ chế bay hơi và tăng cường pheromone từ các đường đi tốt. Vì vậy, bước khởi tạo có thể được xem là trạng thái xuất phát của toàn bộ quá trình học tập tập thể trong thuật toán ACO.

Công thức:

$$\tau_{ij}(0) = \tau_0$$

Trong đó:

- $\tau_{ij}(0)$là lượng pheromone ban đầu trên cạnh nối giữa đỉnh $i$và đỉnh j;
- $\tau_0$là giá trị pheromone khởi tạo chung cho các cạnh.

Bảng 3.2. Các tham số khởi tạo của thuật toán ACO

Tham số	**Ký hiệu**	**Ý nghĩa**	**Giá trị sử dụng**
Số lượng kiến	(m)	Số cá thể kiến tham gia tìm kiếm trong mỗi vòng lặp	15
Số vòng lặp	(N)	Số lần thuật toán lặp để cập nhật pheromone và cải thiện lời giải	60
Hệ số ảnh hưởng pheromone	(α)	Mức độ ảnh hưởng của lượng pheromone đến xác suất chọn cạnh	1.0
Hệ số ảnh hưởng heuristic	(β)	Mức độ ảnh hưởng của thông tin heuristic đến xác suất chọn cạnh	3.0
Hệ số bay hơi pheromone	(ρ)	Tốc độ bay hơi pheromone sau mỗi vòng lặp	0.2
Hằng số cập nhật pheromone	(Q)	Hệ số dùng để tính lượng pheromone bổ sung	100.0
Pheromone ban đầu	(τ)	Lượng pheromone khởi tạo trên các cạnh	1.0
Đỉnh nguồn	(s)	Đỉnh bắt đầu tìm đường	0
Đỉnh đích	(t)	Đỉnh cần đi đến	5
Hạt giống ngẫu nhiên	—	Giá trị khởi tạo để bảo đảm khả năng lặp lại kết quả	7

3.4.3. Mô phỏng quá trình di chuyển của kiến

Sau khi hoàn tất bước khởi tạo, chương trình tiến hành mô phỏng quá trình di chuyển của các cá thể kiến trên đồ thị. Mỗi con kiến bắt đầu từ đỉnh nguồn và từng bước lựa chọn đỉnh tiếp theo cho đến khi đến được đỉnh đích. Quá trình lựa chọn này không mang tính ngẫu nhiên hoàn toàn mà được xác định theo xác suất, dựa trên sự kết hợp giữa lượng pheromone hiện có trên cạnh và thông tin heuristic của cạnh đó.

Trong mỗi bước di chuyển, kiến sẽ xem xét tập các đỉnh kề có thể đi tới từ vị trí hiện tại, đồng thời loại bỏ những đỉnh không phù hợp theo quy tắc của chương trình để tránh lặp không cần thiết. Xác suất lựa chọn một cạnh càng lớn khi cạnh đó có lượng pheromone cao và trọng số nhỏ. Nhờ vậy, các tuyến đường tốt dần trở nên hấp dẫn hơn

qua từng vòng lặp, trong khi thuật toán vẫn duy trì được khả năng khám phá những hướng đi khác.

Quá trình mô phỏng sự di chuyển của kiến chính là phần trung tâm của thuật toán ACO. Mỗi đường đi được một cá thể kiến xây dựng sẽ tạo thành một lời giải ứng viên cho bài toán. Sau khi toàn bộ tập kiến hoàn thành hành trình của mình, chương trình sẽ tính toán độ dài các đường đi tương ứng để phục vụ cho bước cập nhật pheromone và lựa chọn lời giải tốt nhất. Có thể nói, chất lượng của bước mô phỏng di chuyển ảnh hưởng trực tiếp đến khả năng hội tụ và hiệu quả tìm kiếm của toàn bộ thuật toán.

Công thức xác suất:

$$P_{ij}^{k} = \frac{[\tau_{il}]^{\alpha}[\eta_{il}]^{\beta}}{\sum_{l \in N_i^k}[\tau_{il}]^{\alpha}[\eta_{il}]^{\beta}}$$

Trong đó:

- τ_{ij}: lượng pheromone trên cạnh (i, j);
- η_{ij}: thông tin heuristic, thường lấy $\eta_{ij} = \frac{1}{d_{ij}}$;
- α: hệ số ảnh hưởng của pheromone;
- β: hệ số ảnh hưởng của heuristic;
- N_i^k: tập đỉnh mà kiến $k$có thể lựa chọn từ đỉnh i.

3.4.4. Cập nhật pheromone

Sau khi toàn bộ các cá thể kiến hoàn thành đường đi của mình trong một vòng lặp, chương trình thực hiện bước cập nhật pheromone trên các cạnh. Đây là cơ chế quan trọng giúp thuật toán ghi nhận kinh nghiệm từ các lời giải vừa tìm được và định hướng quá trình tìm kiếm ở các vòng lặp tiếp theo. Quá trình cập nhật pheromone bao gồm hai thành phần chính là bay hơi pheromone và tăng cường pheromone trên những cạnh thuộc các đường đi tốt.

Trước hết, lượng pheromone hiện có trên tất cả các cạnh sẽ bị giảm đi một phần theo hệ số bay hơi. Cơ chế này giúp làm mờ dần ảnh hưởng của những lựa chọn cũ, đặc biệt là các đường đi kém hiệu quả, từ đó tránh cho thuật toán bị phụ thuộc quá mức vào các lời giải trước đó. Sau đó, những cạnh nằm trên các đường đi mà kiến vừa đi qua sẽ được cộng thêm một lượng pheromone mới. Đối với các đường đi có tổng chi phí nhỏ, lượng pheromone bổ sung thường lớn hơn, khiến chúng trở nên hấp dẫn hơn trong các vòng lặp sau.

Thông qua quá trình cập nhật này, thuật toán dần dần tăng cường những tuyến đường tốt và giảm vai trò của những tuyến đường không hiệu quả. Chính sự kết hợp giữa bay hơi và tăng cường pheromone đã tạo nên khả năng học tập tập thể của ACO, giúp thuật toán từng bước hội tụ đến đường đi tối ưu hoặc gần tối ưu trên đồ thị.

Công thức**:**

$$\tau_{ij}(t+1) = (1-\rho)\tau_{ij}(t) + \Delta\tau_{ij}$$

Trong đó:

- ρ: hệ số bay hơi pheromone, với $0 < \rho < 1$;
- $\Delta\tau_{ij}$: lượng pheromone được bổ sung trên cạnh (i, j).

Với:

$$\Delta\tau_{ij} = \sum_{k=1}^{m} \Delta\,\tau_{ij}^{k}$$

và

$$\Delta\tau_{ij}^{k} = \begin{cases} \dfrac{Q}{L_k}, & \text{nếu kiến } k \text{ đi qua cạnh } (i,j) \\ 0, & \text{ngược lại} \end{cases}$$

Trong đó:

- m: số lượng kiến;
- Q: hằng số khuếch đại pheromone;
- L_k: độ dài đường đi của kiến thứ k.

3.4.5. Tìm đường đi tốt nhất

Sau mỗi vòng lặp, chương trình tiến hành đánh giá các đường đi do các cá thể kiến xây dựng để xác định lời giải tốt nhất hiện tại. Việc đánh giá này được thực hiện dựa trên tổng trọng số của từng đường đi, trong đó đường đi có chi phí nhỏ nhất sẽ được xem là lời giải tốt nhất ở vòng lặp tương ứng. Nếu lời giải đó tốt hơn kết quả tốt nhất đã lưu trước đó, chương trình sẽ cập nhật lại thông tin về đường đi tốt nhất toàn cục.

Chức năng tìm đường đi tốt nhất đóng vai trò quan trọng trong việc theo dõi quá trình hội tụ của thuật toán. Nhờ việc ghi nhận lời giải tối ưu tạm thời sau từng vòng lặp, chương trình có thể quan sát được xu hướng cải thiện của kết quả theo thời gian. Đồng thời, thông tin này cũng được sử dụng để phục vụ cho phần trực quan hóa và so sánh kết quả thực nghiệm ở các mục tiếp theo.

Khi kết thúc toàn bộ số vòng lặp, đường đi tốt nhất được lưu lại chính là kết quả đầu ra của thuật toán. Đây là tuyến đường mà chương trình đánh giá có tổng chi phí nhỏ nhất trong toàn bộ quá trình thực nghiệm. Thông qua kết quả này, nhóm có thể đối chiếu với dữ liệu đầu vào, kiểm tra tính hợp lý của thuật toán và đưa ra nhận xét về hiệu quả hoạt động của mô hình ACO trong bài toán tìm đường đi ngắn nhất.

3.5. Dữ liệu thử nghiệm

Để kiểm tra khả năng hoạt động của chương trình, đề tài sử dụng bộ dữ liệu thử nghiệm được lưu trong tệp sample_matrix.csv. Dữ liệu đầu vào được biểu diễn dưới dạng ma trận trọng số, trong đó mỗi phần tử của ma trận thể hiện chi phí di chuyển trực tiếp giữa hai đỉnh của đồ thị. Giá trị bằng 0 biểu thị rằng không tồn tại cạnh nối trực tiếp giữa hai đỉnh tương ứng, ngoại trừ các phần tử trên đường chéo chính.

Trong project, chương trình còn hỗ trợ thêm một định dạng dữ liệu khác là danh sách cạnh trong tệp sample_graph.txt. Tuy nhiên, trong quá trình thực nghiệm được trình bày trong báo cáo này, chương trình chính sử dụng dữ liệu từ tệp sample_matrix.csv để xây dựng đồ thị đầu vào và thực hiện mô phỏng thuật toán ACO. Việc lựa chọn dạng ma trận trọng số giúp quá trình xử lý dữ liệu trở nên trực quan, đồng thời thuận tiện cho việc biểu diễn và kiểm tra kết quả.

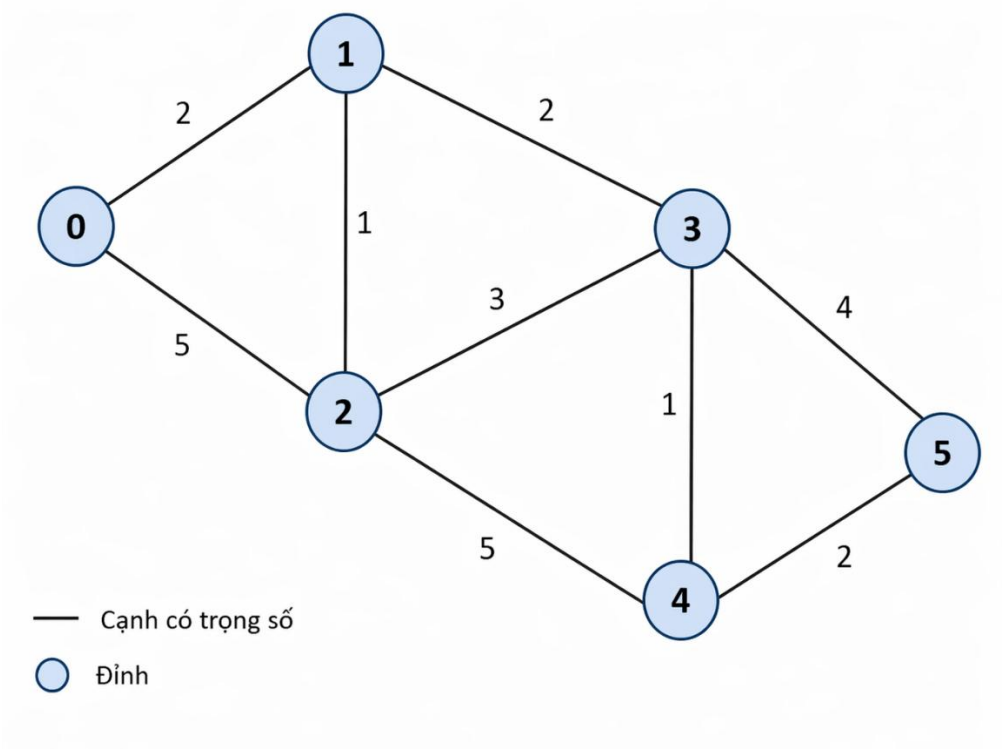
Bộ dữ liệu thử nghiệm gồm 6 đỉnh với các cạnh mang trọng số dương. Từ ma trận đầu vào, chương trình xây dựng đồ thị tương ứng và tiến hành tìm đường đi ngắn nhất từ đỉnh nguồn đến đỉnh đích. Quy mô dữ liệu không quá lớn nhưng đủ để thể hiện rõ cơ chế hoạt động của thuật toán ACO, đồng thời tạo điều kiện thuận lợi cho việc đối chiếu với thuật toán Dijkstra trong phần thực nghiệm.

Bảng 3.3. Ma trận trọng số của đồ thị dùng trong thực nghiệm

Đỉnh	**0**	**1**	**2**	**3**	**4**	**5**
0	0	2	5	0	0	0
1	2	0	1	2	0	0
2	5	1	0	3	5	0
3	0	2	3	0	1	4
4	0	0	5	1	0	2
5	0	0	0	4	2	0

Từ ma trận trên có thể thấy giữa các đỉnh tồn tại nhiều tuyến đường khác nhau từ đỉnh bắt đầu đến đỉnh đích, với tổng trọng số không giống nhau. Điều này cho phép

thuật toán ACO phát huy cơ chế lựa chọn dựa trên xác suất và cập nhật pheromone để từng bước xác định đường đi có chi phí nhỏ nhất.



Hình 3.3 Đồ thị được xây dựng từ dữ liệu đầu vào

(Các số trên cạnh là trọng số giữa các đỉnh)

3.6. Kết quả chạy chương trình

Sau khi hoàn thành việc cài đặt và thiết lập các tham số cho thuật toán, chương trình được thực thi trên bộ dữ liệu thử nghiệm đã trình bày ở mục trước. Kết quả thu được cho thấy thuật toán ACO đã tìm được một đường đi từ đỉnh nguồn đến đỉnh đích với tổng chi phí nhỏ nhất là **7.0000**. Cụ thể, đường đi tốt nhất mà chương trình xác định được là:

$$0 \rightarrow 1 \rightarrow 3 \rightarrow 4 \rightarrow 5$$

Kết quả này cho thấy thuật toán đã hoạt động đúng theo mục tiêu đặt ra, đồng thời chứng minh được khả năng áp dụng của ACO vào bài toán tìm đường đi ngắn nhất trên đồ thị có trọng số. Việc lưu lại đường đi tốt nhất và tổng chi phí tương ứng giúp chương trình có thể tiếp tục phục vụ cho phần trực quan hóa và đánh giá thực nghiệm ở các bước sau.

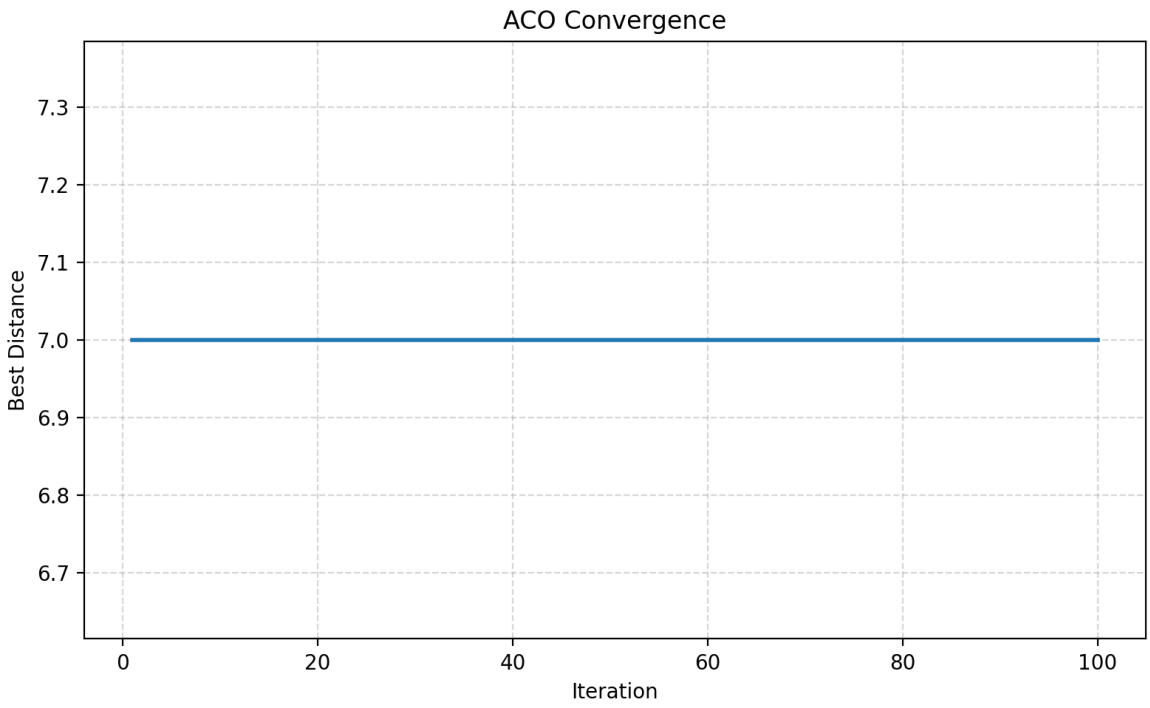
intelligent computing > aco_shortest_path > results > best_path.txt

```
Ant Colony Optimization - Shortest Path Result
Source: 0
Destination: 5
Best path: 0 -> 1 -> 3 -> 4 -> 5
Best distance: 7.0000
```

Hình 3.4. Kết quả đường đi tốt nhất do thuật toán ACO tìm được

Để đánh giá quá trình hội tụ của thuật toán, chương trình đồng thời ghi nhận giá trị lời giải tốt nhất theo từng vòng lặp. Dựa trên biểu đồ hội tụ thu được, có thể thấy giá trị khoảng cách tốt nhất ổn định ở mức **7.0** trong suốt quá trình chạy. Điều này cho thấy với bộ dữ liệu thử nghiệm hiện tại, thuật toán nhanh chóng tìm được lời giải tối ưu và duy trì kết quả ổn định qua các vòng lặp tiếp theo.

Biểu đồ hội tụ là một thành phần quan trọng trong phần thực nghiệm, bởi nó cho phép quan sát xu hướng cải thiện của lời giải theo thời gian. Trong trường hợp này, đường biểu diễn gần như giữ nguyên ở mức tốt nhất đã tìm được, cho thấy thuật toán không bị dao động mạnh và có khả năng hội tụ tốt trên đồ thị mẫu.



Hình 3.5. Biểu đồ hội tụ của thuật toán ACO

Bên cạnh việc đánh giá riêng thuật toán ACO, chương trình còn thực hiện đối chiếu kết quả với thuật toán Dijkstra nhằm kiểm tra độ chính xác của lời giải và so sánh hiệu quả thực thi. Kết quả cho thấy cả hai thuật toán đều tìm được cùng một đường đi là:

$$0 \rightarrow 1 \rightarrow 3 \rightarrow 4 \rightarrow 5$$

với tổng độ dài bằng **7.0000**. Điều này cho thấy trên bộ dữ liệu thử nghiệm, thuật toán ACO đã tìm được lời giải tối ưu tương đương với thuật toán Dijkstra.

Tuy nhiên, về thời gian thực thi, có sự khác biệt khá rõ giữa hai phương pháp. Theo kết quả thu được từ chương trình:

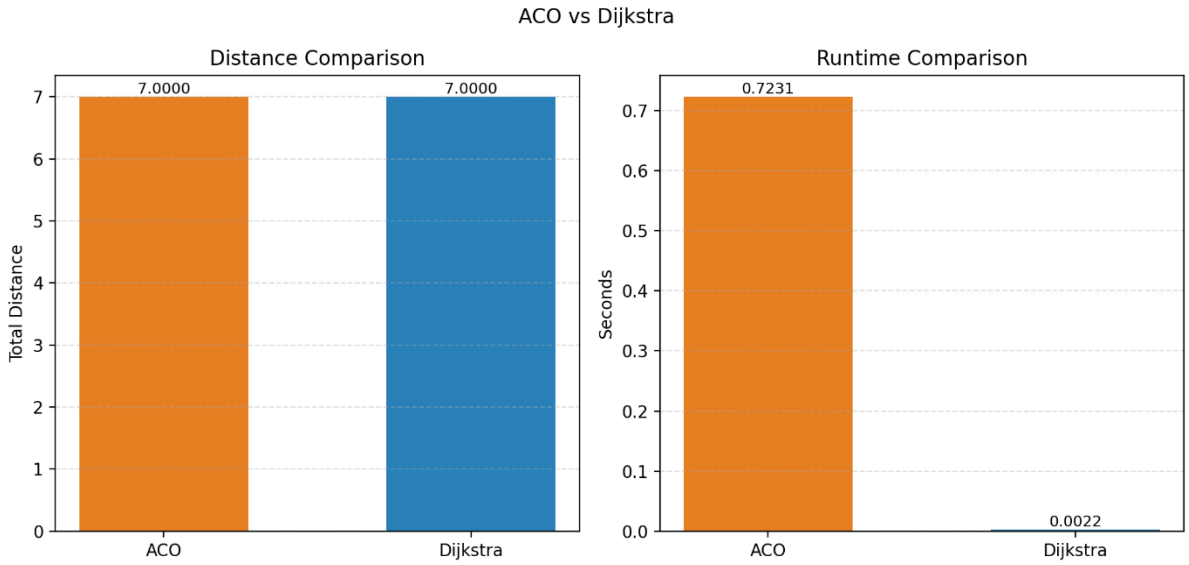
- thời gian chạy của **ACO** là khoảng **0.723141 giây**;
- thời gian chạy của **Dijkstra** là khoảng **0.002187 giây**.

Như vậy, mặc dù ACO đạt được cùng chất lượng lời giải với Dijkstra, nhưng thời gian xử lý trên bộ dữ liệu mẫu lớn hơn đáng kể. Điều này là phù hợp với đặc điểm của hai phương pháp, bởi Dijkstra là thuật toán tất định, được xây dựng chuyên biệt cho bài toán tìm đường đi ngắn nhất, trong khi ACO là thuật toán tối ưu hóa bầy đàn dựa trên cơ chế lặp và mô phỏng xác suất nên cần nhiều thời gian xử lý hơn.

Bảng 3.4. So sánh kết quả giữa ACO và Dijkstra

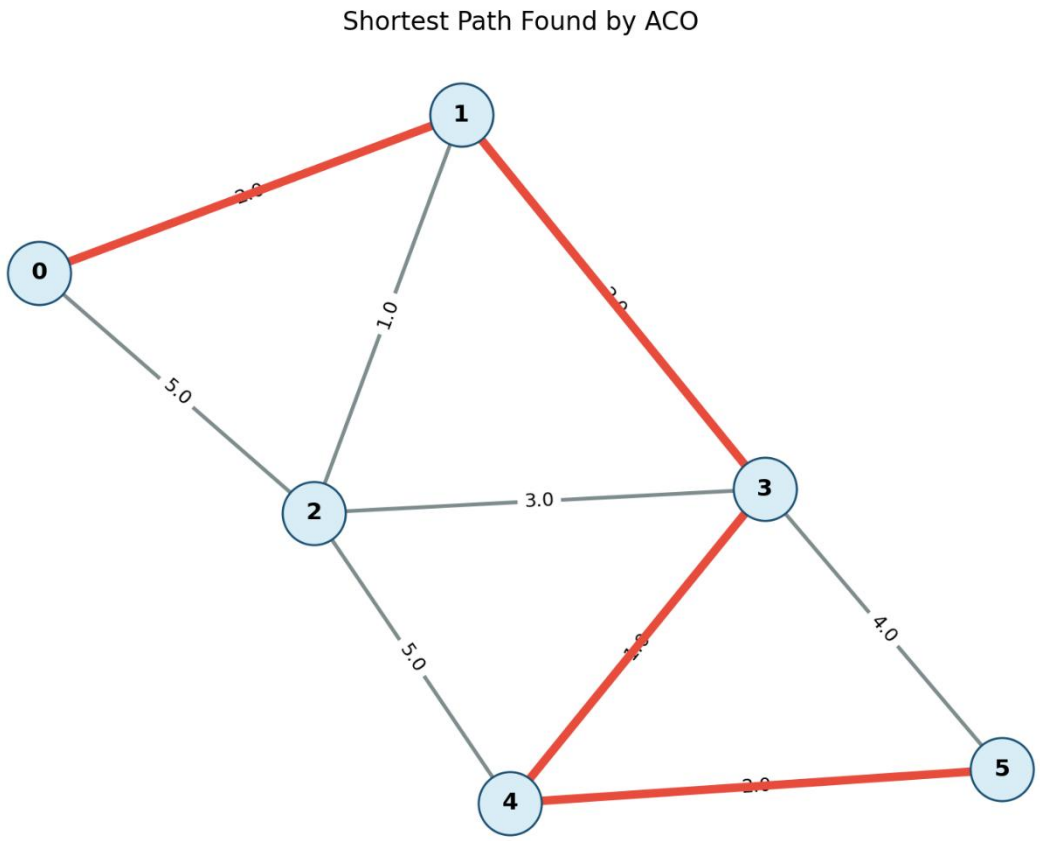
Thuật toán	**Đường đi tìm được**	**Tổng độ dài**	**Thời gian chạy (giây)**
ACO	0 → 1 → 3 → 4 → 5	7.0000	0.723141
Dijkstra	0 → 1 → 3 → 4 → 5	7.0000	0.002187

Từ kết quả trong Bảng 3.4 có thể thấy chênh lệch về độ dài đường đi giữa hai thuật toán bằng 0.0000, đồng thời chất lượng lời giải của ACO đạt 100.00% so với Dijkstra trên đồ thị mẫu. Điều này cho thấy thuật toán ACO trong chương trình đã được cài đặt đúng và có khả năng tìm ra lời giải tối ưu trên dữ liệu thử nghiệm.



Hình 3.6. Biểu đồ so sánh kết quả giữa ACO và Dijkstra

Ngoài ra, để trực quan hóa rõ hơn lời giải tìm được, chương trình còn hiển thị đồ thị cùng với đường đi ngắn nhất được tô nổi bật. Hình ảnh trực quan này cho phép người đọc dễ dàng quan sát các đỉnh, các cạnh và tuyến đường mà thuật toán đã lựa chọn là tối ưu. Đây là một hình thức minh họa hiệu quả, giúp tăng tính thuyết phục cho phần kết quả thực nghiệm.



Hình 3.7. Minh họa đường đi ngắn nhất tìm được trên đồ thị

3.7. Nhận xét kết quả

Từ các kết quả thực nghiệm thu được, có thể nhận thấy chương trình đã thực hiện đúng các chức năng cốt lõi của thuật toán Ant Colony Optimization trong bài toán tìm đường đi ngắn nhất trên đồ thị. Thuật toán không chỉ tìm được một đường đi hợp lệ từ đỉnh nguồn đến đỉnh đích mà còn xác định được đường đi có tổng chi phí nhỏ nhất trên bộ dữ liệu thử nghiệm. Kết quả này cho thấy quá trình mô phỏng sự di chuyển của kiến, cơ chế lựa chọn cạnh theo xác suất và bước cập nhật pheromone đã được triển khai hợp lý.

Một điểm đáng chú ý là thuật toán ACO cho kết quả trùng khớp với thuật toán Dijkstra về cả đường đi lẫn tổng độ dài. Cụ thể, cả hai phương pháp đều tìm được đường đi ngắn nhất là $0 \rightarrow 1 \rightarrow 3 \rightarrow 4 \rightarrow 5$ với tổng độ dài bằng 7.0000. Khoảng cách chênh

lệch giữa hai thuật toán bằng 0.0000, đồng thời chất lượng lời giải của ACO đạt 100.00%so với Dijkstra trên đồ thị mẫu. Điều này cho thấy chương trình đã được cài đặt đúng và thuật toán ACO có khả năng tìm ra lời giải tối ưu trên dữ liệu thử nghiệm.

Tuy nhiên, xét về thời gian thực thi, có sự khác biệt khá rõ giữa hai phương pháp. Theo kết quả chạy chương trình, thời gian thực thi của ACO là khoảng 0.723141giây, trong khi thời gian chạy của Dijkstra chỉ vào khoảng 0.002187giây. Như vậy, mặc dù đạt được cùng chất lượng lời giải, ACO cần thời gian xử lý lớn hơn đáng kể so với Dijkstra trên bộ dữ liệu mẫu.

Sự khác biệt này là phù hợp với bản chất của hai phương pháp. Dijkstra là thuật toán tất định, được thiết kế chuyên biệt cho bài toán tìm đường đi ngắn nhất nên có ưu thế rõ rệt về tốc độ trên các đồ thị nhỏ và tĩnh. Ngược lại, ACO là thuật toán tối ưu hóa bầy đàn dựa trên cơ chế mô phỏng xác suất, cần nhiều vòng lặp, nhiều cá thể kiến và quá trình cập nhật pheromone liên tục, do đó chi phí tính toán thường cao hơn. Mặc dù vậy, ACO vẫn có ý nghĩa quan trọng về mặt nghiên cứu và ứng dụng, đặc biệt khi mở rộng sang các bài toán tối ưu tổ hợp phức tạp hơn hoặc các môi trường có tính động.

Nhìn chung, kết quả thực nghiệm cho thấy chương trình đã đạt được mục tiêu đề ra: cài đặt thành công thuật toán ACO, tìm được đường đi ngắn nhất trên đồ thị mẫu và kiểm chứng tính chính xác của lời giải thông qua việc so sánh với thuật toán Dijkstra. Đây là cơ sở quan trọng để khẳng định tính khả thi của hướng tiếp cận sử dụng ACO trong bài toán đã nghiên cứu.

3.8. Kết luận chương 3

Trong chương này, báo cáo đã trình bày quá trình cài đặt chương trình mô phỏng thuật toán Ant Colony Optimization cho bài toán tìm đường đi ngắn nhất trên đồ thị bằng ngôn ngữ Python. Các nội dung chính bao gồm môi trường cài đặt, công cụ và thư viện sử dụng, cấu trúc tổng thể của chương trình, cùng với việc hiện thực các chức năng quan trọng như nhập dữ liệu đồ thị, khởi tạo pheromone, mô phỏng quá trình di chuyển của kiến, cập nhật pheromone và tìm đường đi tốt nhất.

Bên cạnh phần cài đặt, chương cũng đã trình bày dữ liệu thử nghiệm và các kết quả thu được khi chạy chương trình. Kết quả cho thấy thuật toán ACO đã tìm được đường đi ngắn nhất $0 \to 1 \to 3 \to 4 \to 5$với tổng chi phí bằng 7.0000. Đồng thời, khi đối chiếu với thuật toán Dijkstra, chương trình cho thấy cả hai phương pháp đều thu được cùng một lời giải tối ưu trên đồ thị mẫu. Điều này góp phần khẳng định tính đúng đắn của thuật toán đã cài đặt.

Ngoài ra, chương cũng cho thấy mặc dù ACO đạt được chất lượng lời giải tương đương với Dijkstra, thời gian thực thi của ACO vẫn lớn hơn đáng kể trên bộ dữ liệu thử nghiệm. Kết quả này phản ánh đúng đặc trưng của một thuật toán tối ưu hóa bầy đàn, trong đó chất lượng lời giải đạt được thông qua quá trình lặp và trao đổi thông tin gián tiếp giữa các cá thể. Từ đó có thể thấy rằng ACO không chỉ có giá trị trong việc giải quyết bài toán cụ thể của đề tài mà còn là cơ sở để tiếp cận các bài toán tối ưu phức tạp hơn trong lĩnh vực tính toán thông minh.

Như vậy, chương 3 đã hoàn thành vai trò kiểm chứng thực nghiệm cho các nội dung lý thuyết và mô hình thuật toán được trình bày ở các chương trước. Đây là nền tảng để báo cáo chuyển sang phần kết luận chung, trong đó tổng hợp những kết quả đã đạt được, chỉ ra các hạn chế còn tồn tại và đề xuất hướng phát triển tiếp theo cho đề tài.

CHƯƠNG 4: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

4.1. Kết quả đạt được

Sau quá trình nghiên cứu, xây dựng và cài đặt chương trình, đề tài "Xây dựng thuật toán Ant Colony Optimization cho bài toán tìm đường đi ngắn nhất trên đồ thị" đã đạt được những kết quả chính sau đây.

Trước hết, đề tài đã hệ thống hóa được những cơ sở lý thuyết cần thiết liên quan đến tính toán thông minh, lý thuyết đồ thị, bài toán tìm đường đi ngắn nhất và thuật toán Ant Colony Optimization. Thông qua việc tìm hiểu nguyên lý hoạt động của đàn kiến trong tự nhiên, cơ chế pheromone, quy tắc lựa chọn đường đi và phương pháp cập nhật pheromone, báo cáo đã làm rõ được cách thức áp dụng ACO vào bài toán tối ưu trên đồ thị có trọng số.

Tiếp theo, đề tài đã xây dựng được mô hình thuật toán ACO phù hợp với bài toán tìm đường đi ngắn nhất giữa một đỉnh nguồn và một đỉnh đích xác định. Các thành phần quan trọng của thuật toán như biểu diễn dữ liệu đồ thị, khởi tạo pheromone, tính xác suất chọn đỉnh tiếp theo, mô phỏng quá trình di chuyển của kiến và cập nhật pheromone sau mỗi vòng lặp đều đã được trình bày và hiện thực hóa trong chương trình.

Về mặt cài đặt, nhóm đã xây dựng thành công chương trình mô phỏng bằng ngôn ngữ Python với cấu trúc rõ ràng, có khả năng tiếp nhận dữ liệu đầu vào, thực hiện thuật toán ACO và trực quan hóa kết quả. Chương trình không chỉ tìm được đường đi tốt nhất trên đồ thị thử nghiệm mà còn cho phép theo dõi quá trình hội tụ của thuật toán qua biểu đồ và hình minh họa trực quan.

Kết quả thực nghiệm cho thấy thuật toán ACO đã tìm được đường đi ngắn nhất từ đỉnh nguồn đến đỉnh đích là:

$$0 \rightarrow 1 \rightarrow 3 \rightarrow 4 \rightarrow 5$$

với tổng độ dài bằng **7.0000**. Đồng thời, khi đối chiếu với thuật toán Dijkstra, kết quả cho thấy cả hai phương pháp đều tìm được cùng một lời giải tối ưu trên đồ thị mẫu. Điều này chứng tỏ chương trình đã được cài đặt đúng và thuật toán ACO có khả năng giải quyết hiệu quả bài toán tìm đường đi ngắn nhất trong phạm vi dữ liệu thử nghiệm đã lựa chọn.

Nhìn chung, đề tài đã hoàn thành được mục tiêu đặt ra ban đầu, bao gồm nghiên cứu cơ sở lý thuyết, xây dựng thuật toán, cài đặt chương trình và đánh giá kết quả thực nghiệm.

4.2. Hạn chế của chương trình

Mặc dù đã đạt được những kết quả nhất định, chương trình vẫn còn một số hạn chế cần được nhìn nhận rõ.

Thứ nhất, chương trình hiện mới được thử nghiệm trên đồ thị có kích thước nhỏ và dữ liệu đầu vào tương đối đơn giản. Điều này giúp thuận lợi cho việc mô phỏng và kiểm tra tính đúng đắn của thuật toán, nhưng chưa phản ánh đầy đủ hiệu quả của ACO khi áp dụng trên các đồ thị lớn, phức tạp hoặc có nhiều ràng buộc hơn trong thực tế.

Thứ hai, việc lựa chọn các tham số của thuật toán như số lượng kiến, số vòng lặp, hệ số α, β, hệ số bay hơi ρvà hằng số Q hiện vẫn mang tính thủ công. Hiệu quả của thuật toán phụ thuộc khá nhiều vào các tham số này, trong khi chương trình chưa có cơ chế tự động tinh chỉnh hoặc tối ưu tham số để đạt hiệu suất tốt nhất trong từng trường hợp cụ thể.

Thứ ba, so với thuật toán Dijkstra, thời gian thực thi của ACO trên bộ dữ liệu mẫu còn lớn hơn đáng kể. Đây là hạn chế dễ thấy khi áp dụng ACO cho bài toán đường đi ngắn nhất cơ bản, bởi phương pháp này cần trải qua nhiều vòng lặp và quá trình mô phỏng xác suất phức tạp hơn. Do đó, trong những bài toán nhỏ và tĩnh, ACO chưa thể hiện ưu thế về tốc độ so với các thuật toán tất định truyền thống.

Thứ tư, chương trình hiện chủ yếu tập trung vào việc tìm đường đi ngắn nhất trên đồ thị tĩnh với trọng số dương, chưa xét đến các trường hợp mở rộng như đồ thị thay đổi theo thời gian, trọng số động, bài toán đa mục tiêu hoặc các ràng buộc thực tế khác như giới hạn tài nguyên, ưu tiên tuyến đường hay điều kiện di chuyển.

Ngoài ra, phần giao diện của chương trình còn ở mức đơn giản, chủ yếu phục vụ cho mục đích mô phỏng và kiểm tra kết quả. Chương trình chưa được phát triển thành một hệ thống hoàn chỉnh có khả năng tương tác trực quan và thuận tiện hơn cho người dùng cuối.

4.3. Hướng phát triển

Từ những kết quả đạt được và các hạn chế còn tồn tại, đề tài có thể được tiếp tục phát triển theo một số hướng sau.

Trước hết, chương trình có thể được mở rộng để thử nghiệm trên các đồ thị có kích thước lớn hơn và cấu trúc phức tạp hơn. Việc này sẽ giúp đánh giá rõ hơn khả năng hội tụ, độ ổn định và hiệu quả thực tế của thuật toán ACO trong những môi trường gần với bài toán ứng dụng hơn.

Tiếp theo, một hướng phát triển quan trọng là nghiên cứu cơ chế lựa chọn hoặc tối ưu tham số tự động cho thuật toán. Việc tinh chỉnh tham số phù hợp có thể giúp cải thiện đáng kể chất lượng lời giải và thời gian hội tụ, đồng thời làm cho chương trình thích nghi tốt hơn với nhiều loại dữ liệu đầu vào khác nhau.

Bên cạnh đó, chương trình có thể được mở rộng để xử lý các biến thể khó hơn của bài toán, chẳng hạn như bài toán tìm đường đi ngắn nhất trong môi trường động, bài toán có nhiều tiêu chí tối ưu đồng thời, hoặc bài toán định tuyến có thêm các ràng buộc thực tế. Đây là những hướng nghiên cứu có giá trị ứng dụng cao trong các lĩnh vực như giao thông thông minh, logistics, mạng máy tính và robot tự hành.

Một hướng khác là kết hợp ACO với các phương pháp tối ưu khác nhằm nâng cao hiệu quả tìm kiếm. Chẳng hạn, có thể nghiên cứu các mô hình lai giữa ACO và thuật toán di truyền, tối ưu bầy đàn hạt hoặc các thuật toán tìm kiếm cục bộ để tận dụng ưu điểm của từng phương pháp.

Ngoài ra, phần trực quan hóa và giao diện chương trình cũng có thể được phát triển thêm để hỗ trợ người dùng dễ dàng nhập dữ liệu, thay đổi tham số và theo dõi kết quả một cách trực quan hơn. Nếu được mở rộng theo hướng này, chương trình không chỉ có giá trị học thuật mà còn có thể trở thành một công cụ hỗ trợ thực nghiệm và giảng dạy hiệu quả.

Tóm lại, đề tài đã đặt được nền tảng ban đầu cho việc nghiên cứu và ứng dụng thuật toán Ant Colony Optimization vào bài toán tìm đường đi ngắn nhất trên đồ thị. Trong tương lai, nếu tiếp tục được hoàn thiện và mở rộng, chương trình có thể phát triển theo hướng có tính ứng dụng cao hơn, đồng thời góp phần làm rõ hơn tiềm năng của các phương pháp tối ưu hóa bầy đàn trong lĩnh vực tính toán thông minh.

TÀI LIỆU THAM KHẢO

[1] Marco Dorigo and Thomas Stützle, *Ant Colony Optimization*, MIT Press, 2004.

[2] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, *Introduction to Algorithms*, 3rd Edition, MIT Press, 2009.

[3] Nguyễn Đức Nghĩa, *Lý thuyết đồ thị và ứng dụng*, NXB Đại học Quốc gia Hà Nội.

PHỤ LỤC

Phụ lục A. Mã nguồn chương trình

Phụ lục này trình bày mã nguồn chính của chương trình cài đặt thuật toán Ant Colony Optimization cho bài toán tìm đường đi ngắn nhất trên đồ thị. Nhằm thuận tiện cho việc kiểm tra và đối chiếu mã nguồn, toàn bộ chương trình của đề tài được lưu trữ tại kho GitHub sau: https://github.com/ntthach-hub/SGU/tree/main/intelligent%20computing/aco_shortest_path